PySpark — Complete Handbook

Spark architecture, DataFrame API, joins, partitioning, internals, tuning and Structured Streaming

notes by Sagar · Jan 2026

Contents

1. How to use this handbook
2. What Spark is and why it exists
3. Setting up and your first program
4. Architecture
5. Core concepts
6. SparkSession and configuration
7. Reading data
8. Writing data
9. Schemas and data types
10. DataFrame transformations
11. Built-in functions
12. Aggregations
13. Joins
14. Window functions
15. Spark SQL
16. UDFs and pandas integration
17. Partitioning
18. Caching and persistence
19. Catalyst, Tungsten and Adaptive Query Execution
20. Memory and cluster sizing
21. Performance playbook
22. Reading the Spark UI
23. Structured Streaming
24. Testing and data quality
25. Common pipeline patterns
26. Submitting and deploying Spark jobs
27. Common beginner mistakes
28. Cluster sizing scenarios: 10 GB to 2 TB per day
29. Practice project: an end-to-end mini pipeline
30. PySpark cheat sheet
31. Remember it for life: memory hooks
32. Glossary

Sagar

1. How to use this handbook

This handbook takes you from "what is Spark?" to designing and tuning production pipelines. You don't need to read it in one go, and you don't need to understand every chapter on day one.

Learning path

Stage	Read	Goal
Week 1 — foundations	Chapters 2–5 (What Spark is, Setup, Architecture, Core concepts)	Understand driver, executors, partitions, lazy evaluation, shuffle. Run your first job
Week 2 — everyday work	Chapters 6–13 (Session, Reading, Writing, Schemas, Transformations, Functions, Aggregations, Joins)	Write real transformations confidently
Week 3 — intermediate	Chapters 14–17 (Windows, Spark SQL, UDFs, Partitioning)	Solve most interview coding questions and pipeline tasks
Week 4 — performance	Chapters 18–22 (Caching, Catalyst/AQE, Memory, Performance playbook, Spark UI)	Explain *why* a job is slow and fix it
Week 5 — production	Chapters 23–26 (Streaming, Testing, Pipeline patterns, Deployment)	Build reliable, idempotent pipelines
Before interviews	Chapters 27–32 (Beginner mistakes, Sizing, Practice project, Cheat sheet, Memory hooks, Glossary)	Revise fast, practise, and answer sizing questions

How each chapter is written

- Every important idea starts with a **simple explanation**, then the technical detail.
- Boxes like this one hold the key takeaway or a warning — if you only read the boxes, you still get the main points.
- **Memory hooks** in the boxes (stories, images and short mnemonics) help you recall each idea at a glance. Chapter 31 collects them all with a self-test and a revision plan.
- Code is minimal and runnable. Type it out yourself instead of copying; it sticks better.

> First time through, focus on the **why**. Spark makes sense once you understand three ideas: data is split into partitions, nothing runs until an action, and moving data between machines (shuffle) is the expensive part. Everything else builds on those.

Imports used throughout

```
from pyspark.sql import SparkSession, functions as F, Window as W
from pyspark.sql.types import *
from pyspark import StorageLevel
```

2. What Spark is and why it exists

Apache Spark is a distributed engine for processing large data across a cluster of machines. You write code once; Spark splits the data into partitions and runs the work in parallel on many cores.

In simple words: the restaurant kitchen

Imagine you have to chop 10,000 kg of vegetables.

- **One cook** (one computer) would take weeks. That's pandas or a plain Python script on big data.
- **A kitchen with a head chef and 100 cooks** finishes in hours. The head chef (the **driver**) reads the recipe (your code), splits the vegetables into baskets (**partitions**), and hands baskets to cooks (**executors**). Each cook works on their own basket at the same time.
- When the recipe says "put all the onions together", cooks have to walk around and swap ingredients. That swapping is the **shuffle** — it's the slowest part, so good recipes avoid it.
- If a cook falls sick, the head chef gives their basket to someone else, because the recipe (the **lineage**) says exactly how to redo it.

That's Spark: split the data, process the pieces in parallel, move data around only when needed, and recover from failures automatically.

Why not just one machine?

When data is bigger than one machine's memory, or processing takes too long on one machine, you need to split the work. Spark handles splitting, scheduling, moving data between machines, and recovering from failures — you describe *what* to compute.

Spark vs Hadoop MapReduce

	MapReduce	Spark
Intermediate data	Written to disk between every step	Kept in memory where possible
Programming model	Map and reduce functions only	Rich APIs: DataFrames, SQL, streaming
Speed	Slow for multi-step and iterative jobs	Much faster, especially multi-step
Optimizer	None	Catalyst query optimizer
Languages	Java mostly	Python, Scala, Java, SQL, R

Spark components

- **Spark Core:** scheduling, memory, fault tolerance, RDDs.
- **Spark SQL / DataFrames:** structured data, the API you use most.
- **Structured Streaming:** streaming with the same DataFrame API.
- **MLlib:** distributed machine learning.

- **GraphX:** graph processing (Scala).

PySpark

PySpark is the Python API for Spark. Your Python code builds a plan; the plan runs on the JVM-based Spark engine. As long as you use built-in DataFrame functions, Python adds almost no overhead.

When to use Spark — and when not to

Use Spark when	Don't use Spark when
Data is larger than one machine's memory (tens of GB and up)	Data is small (a few hundred MB) — pandas, Polars or DuckDB are simpler and faster
Work must be spread over many cores/machines to meet a deadline	You need millisecond responses for single-record lookups — use a database
You're building lakehouse pipelines (Delta/Iceberg)	A warehouse SQL query already does the job well
You need batch and streaming with the same code	You need a quick one-off script on a CSV

Sagar

3. Setting up and your first program

You learn Spark fastest by running code. There are two easy ways to start.

Option 1: run PySpark on your laptop

```
# needs Java installed (Java 17 is the safe choice; PySpark 4.x requires 17+): check with  java -version
python -m venv .venv && source .venv/bin/activate
pip install pyspark
```

```
# first_job.py
from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder.master("local[*]").appName("first_job").getOrCreate()
# local[*] = run on this machine using all CPU cores (each core acts like a mini executor)

data = [(1, "Asha", "Bengaluru", 1200.0),
        (2, "Ravi", "Pune",       300.0),
        (3, "Meena","Bengaluru",  850.0),
        (4, "Arjun","Delhi",      2100.0)]
df = spark.createDataFrame(data, "id INT, name STRING, city STRING, amount DOUBLE")

df.show()                                  # action: prints the table
df.printSchema()

result = (df.filter(F.col("amount") > 500)   # transformation (lazy)
            .groupBy("city")                 # transformation (lazy)
            .agg(F.sum("amount").alias("total"), F.count("*").alias("orders")))
result.show()                              # action: now Spark actually runs

result.write.mode("overwrite").parquet("output/city_totals")
spark.stop()
```

Run it with **python first_job.py**. While it runs, open **http://localhost:4040** to see the Spark UI — jobs, stages and tasks for your program.

Option 2: Databricks Free Edition

Databricks offers a free edition (the successor to the old Community Edition) where you get a workspace with notebooks and serverless compute. **spark** is already created for you — just write DataFrame code in a notebook cell. It's the closest thing to a real job environment for practising Delta Lake, SQL and pipelines.

Sagar

Reading your first output

```
+---------+------+------+
|     city| total|orders|
+---------+------+------+
|Bengaluru|2050.0|     2|
|    Delhi|2100.0|     1|
+---------+------+------+
```

Ravi's order (300) was filtered out; Bengaluru's two orders were summed. (Row order after a **groupBy** isn't guaranteed — add **orderBy** if you need a fixed order.) Notice nothing happened when you wrote **filter** and **groupBy** — Spark only ran when you called **show()**. That's **lazy evaluation**, explained in chapter 5.

> Practise every chapter by running its examples locally on small data. The code is the same on a laptop and on a 100-node cluster — only the data size and configuration change.

Sagar

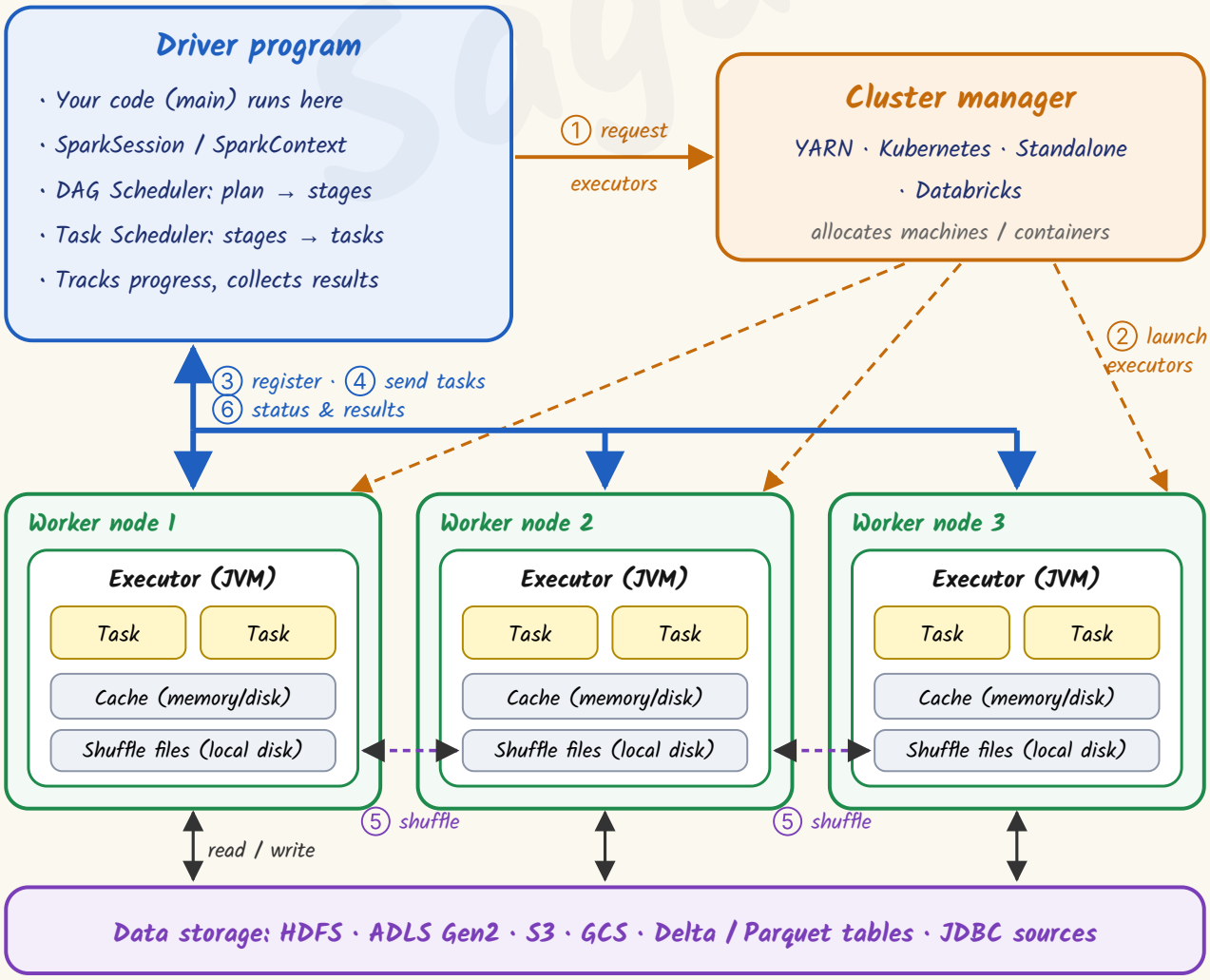
4. Architecture

In simple words: *the* **driver** *is the manager that plans the work;* **executors** *are the workers that do it; the* **cluster manager** *hires the workers. Your code runs on the driver as a plan, and the heavy lifting happens on executors, one* **task** *per piece of data.*

Memory hook — D·C·E·T, "Don't Cook Everything Together": *Driver plans,* **C***luster manager hires,* **E***xecutors cook,* **T***asks are the individual dishes.*

The pieces

Component	Role
Driver	*Runs your program (**main**), creates the SparkSession, builds the plan, splits it into tasks, schedules them, collects results*
Executors	*JVM processes on worker nodes that run tasks and hold cached data*
Cluster manager	*Allocates resources: YARN, Kubernetes, Spark Standalone, or Databricks' own manager*
Tasks	*The smallest unit of work: one task processes one partition*



How a Spark application runs, step by step

1. **Request resources:** you submit the application; the driver starts and asks the cluster manager for executors (how many, how many cores, how much memory).
2. **Launch executors:** the cluster manager starts executor processes (JVMs) on worker nodes. Its job ends there — it does not run your code or move your data.
3. **Register:** each executor connects back to the driver. From now on, the driver and executors talk **directly**.
4. **Plan and send tasks:** when your code calls an action, the driver turns the plan into stages (DAG scheduler) and tasks (task scheduler) and sends each task to an executor — preferably one close to that task's data.
5. **Run tasks:** executors read their partitions from storage, process them, keep cached data, and write shuffle files to local disk. In the next stage, executors fetch shuffle data **from each other**.
6. **Report back:** executors send task status, metrics and small results (like a **count**) to the driver. Large outputs are written straight to storage by the executors — they never pass through the driver.

> Key points interviewers check: the driver plans and schedules but doesn't process the data; executors do the work and talk to each other during shuffles; the cluster manager only hands out resources. If the driver dies, the application dies; if an executor dies, its tasks are re-run elsewhere using lineage.

Jobs, stages and tasks

- **Job:** triggered by each action (**count**, **write**, **collect**).
- **Stage:** a set of tasks that can run without moving data between executors. A **shuffle** ends a stage.
- **Task:** one per partition per stage.

A job reading 200 partitions, filtering, then doing a **groupBy** has two stages: stage 1 (read + filter + partial aggregate, 200 tasks) and stage 2 (final aggregate after shuffle, **spark.sql.shuffle.partitions** tasks).

Deploy modes

- **Client mode:** the driver runs where you submitted the job (your laptop, an edge node). Good for interactive work.
- **Cluster mode:** the driver runs inside the cluster. Good for production — the job doesn't depend on the submitting machine staying alive.

Fault tolerance

Spark records the **lineage** — the chain of transformations that produced each partition. If an executor dies, Spark recomputes only the lost partitions from their lineage. Shuffle files and cached data on a lost executor are recomputed too.

5. Core concepts

RDD, DataFrame, Dataset

Spark has three APIs for distributed data. They all run on the same engine, but they differ in how much Spark knows about your data — and that decides how much it can optimize.

What each one is

- **RDD (Resilient Distributed Dataset)** — the original, low-level API (Spark 1.0). A distributed collection of objects of any type. Spark sees each record as an opaque object: it knows nothing about columns or types, so it runs your functions exactly as written.
- **DataFrame** (Spark 1.3+) — a distributed table with named columns and a schema, like a SQL table or a pandas DataFrame spread across a cluster. Because Spark knows the schema and the operations, the Catalyst optimizer can rewrite and optimize the whole plan.
- **Dataset** (Spark 1.6+, unified with DataFrame in 2.0) — a DataFrame with compile-time types in Scala/Java: **Dataset[Order]** instead of rows. In Scala, a DataFrame is literally **Dataset[Row]**. **Datasets don't exist in Python** — Python is dynamically typed, so PySpark has only RDDs and DataFrames.

Sagar

Side-by-side comparison

Feature	RDD	DataFrame	Dataset
Introduced	Spark 1.0	Spark 1.3	Spark 1.6 (merged with DataFrame in 2.0)
Data representation	Distributed objects (any Python/Java/Scala objects)	Rows with named columns and a schema	Typed JVM objects (case classes / Java beans) with a schema
Schema	None — Spark doesn't know the structure	Yes — known to Spark	Yes — known to Spark and the compiler
Type safety	Compile-time in Scala/Java (runtime in Python)	Runtime: a wrong column name fails when the query is analyzed	**Compile-time**: wrong field names/types fail before the job runs
Optimization	None — runs your lambdas as written	**Catalyst optimizer + Tungsten**	Catalyst + Tungsten (lambdas are opaque, so less than pure DataFrame ops)
Memory / serialization	Java/Python objects; Java or Kryo serialization; heavy GC	Tungsten compact binary format, off-heap; very efficient	Tungsten format with **encoders** for typed objects
Performance	Slowest for structured data (especially in Python)	Fastest; Python ≈ Scala speed with built-in functions	Close to DataFrame; typed lambdas can be slower than built-ins
API style	Functional: **map**, **filter**, **reduceByKey**, **flatMap**	Declarative: **select**, **filter**, **groupBy**, SQL	Both: typed functional + DataFrame ops
SQL support	No (convert to DataFrame first)	Yes — temp views, **spark.sql**	Yes
Languages	Python, Scala, Java	Python, Scala, Java, R, SQL	**Scala and Java only**
Fault tolerance	Lineage-based recomputation	Lineage-based (built on RDDs underneath)	Same
Lazy evaluation	Yes	Yes	Yes
Best for	Unstructured data, custom low-level control, legacy code	Almost all ETL, analytics and SQL work	Scala/Java teams wanting compile-time safety with structured data

The same job in each API

```
# RDD (PySpark): Spark can't see inside the lambdas, no optimization
rdd = spark.sparkContext.textFile("/data/orders.csv")
totals = (rdd.map(lambda line: line.split(","))
             .filter(lambda f: f[3] == "PAID")
             .map(lambda f: (f[1], float(f[2])))          # (customer_id, amount)
             .reduceByKey(lambda a, b: a + b))

# DataFrame (PySpark): declarative, optimized by Catalyst
df = spark.read.option("header", True).schema("order_id INT, customer_id STRING, amount
DOUBLE, status STRING").csv("/data/orders.csv")
totals = df.filter(F.col("status") ==
"PAID").groupBy("customer_id").agg(F.sum("amount").alias("total"))
```

```
// Dataset (Scala only): typed objects, compile-time checks
case class Order(order_id: Int, customer_id: String, amount: Double, status: String)
val ds = spark.read.option("header", "true").schema(schema).csv("/data/orders.csv").as[Order]
val totals = ds.filter(_.status ==
"PAID").groupByKey(_.customer_id).mapValues(_.amount).reduceGroups(_ + _)
// ds.filter(_.stauts == "PAID")  ← typo caught at compile time
```

Why DataFrames are faster than RDDs

1. **Catalyst sees the whole query** — it pushes filters down to the file scan, prunes unused columns, reorders joins and picks broadcast vs sort-merge joins. With RDD lambdas, Spark can't see what your code does, so it can't optimize it.
2. **Tungsten binary storage** — rows are stored compactly in binary, often off-heap, instead of as Java/Python objects. Less memory, less garbage collection.
3. **Whole-stage code generation** — several operators are compiled into one tight Java function.
4. **No Python round-trips** — DataFrame operations run inside the JVM. PySpark RDD operations send every record to a Python worker process and back (serialization both ways), which is why PySpark RDD code is much slower than PySpark DataFrame code.

Converting between them

```
rdd = df.rdd                                  # DataFrame → RDD of Row objects
df2 = rdd.toDF(["customer_id", "total"])      # RDD of tuples → DataFrame
df3 = spark.createDataFrame(rdd, schema)      # with an explicit schema
```

```
val ds = df.as[Order]     // DataFrame → Dataset
val df = ds.toDF()        // Dataset → DataFrame
val rdd = ds.rdd          // Dataset → RDD
```

Converting a DataFrame to an RDD leaves the optimizer behind for everything after that point — avoid it in the middle of a pipeline.

When you'd still use RDDs

- Truly unstructured data that can't be described with a schema (rare — even raw text is usually read as a DataFrame with **spark.read.text**).
- Fine-grained control over partitioning (custom partitioners) or low-level operations (**mapPartitions** with complex per-partition setup — though **mapInPandas** usually covers this now).
- Maintaining legacy Spark 1.x code.

> Short answer: RDD = low-level objects with no schema and no optimizer; DataFrame = schema-aware table optimized by Catalyst and Tungsten — the default for everything; Dataset = typed DataFrame for Scala/Java with compile-time safety, not available in PySpark.

Lazy evaluation

> **In simple words:** writing transformations is like writing a shopping list — nothing is bought yet. Only when you go to the shop (an **action** like **count**, **show** or **write**) does the work happen, and because Spark sees the whole list first, it plans the shortest route.
>
> **Memory hook:** Spark is a lazy genius — it plans everything and does nothing until you say "show me". **Actions** are the words that wake it up: "Can Charlie Show The Work First?" = **C**ount, **C**ollect, **S**how, **T**ake, **W**rite, **F**irst.

Transformations (**select**, **filter**, **join**, **groupBy**) only build a plan. Nothing runs until an **action** asks for a result. This lets Spark optimize the whole plan at once — combining filters, pruning columns, choosing join strategies.

```
df = spark.read.parquet("/data/orders")          # nothing read yet (just metadata)
big = df.filter("amount > 1000").select("order_id", "amount")   # still nothing
big.count()                                      # action → job runs now
```

Transformations vs actions

Transformations (lazy)	Actions (run a job)
select, **withColumn**, **filter**, **where**	**count**, **collect**, **take**, **first**, **show**
join, **groupBy().agg()**, **union**	**write...save/saveAsTable**
orderBy, **distinct**, **dropDuplicates**	**foreach**, **foreachPartition**
repartition, **coalesce**	**toPandas**

Narrow vs wide transformations

- **Narrow:** each output partition depends on one input partition — **filter**, **select**, **withColumn**, **map**. No data moves between executors.
- **Wide:** output partitions need data from many input partitions — **groupBy**, **join** (non-broadcast), **distinct**, **orderBy**, **repartition**. These cause a **shuffle**.

Shuffle

In simple words: *a shuffle is when every machine must send some of its data to every other machine — like 100 cooks each sorting their vegetables and walking them over to the right station. It's needed for* ***groupBy***, ***join***, ***distinct*** *and* ***orderBy***, *and it's usually the slowest part of a job.*

Memory hook — "Just Getting Data Organized Requires a shuffle": *Join, GroupBy, Distinct, OrderBy, Repartition. Narrow stays in its* **lane**; *wide* **walks** *across the kitchen.*

A shuffle redistributes data across the cluster by key: every executor writes its data split by target partition to local disk, then every executor reads its share over the network. It is the most expensive operation in Spark — disk I/O, network I/O and serialization. Most tuning is about avoiding or shrinking shuffles.

Partitions

In simple words: *a partition is one basket of rows. One task processes one basket on one core. 1,000 baskets and 100 cores means 10 rounds ("waves") of work.*

Memory hook — the Goldilocks basket: *not too small (too many tiny tasks), not too big (out of memory) — about* **128 MB** *when reading,* **100–200 MB** *after a shuffle.*

A partition is a chunk of data processed by one task. Parallelism = number of partitions running at once, limited by total executor cores. Too few partitions → idle cores and huge tasks. Too many → scheduling overhead and tiny files.

6. SparkSession and configuration

Creating a session

```
from pyspark.sql import SparkSession

spark = (SparkSession.builder
         .appName("orders_daily")
         .config("spark.sql.shuffle.partitions", "200")
         .config("spark.sql.adaptive.enabled", "true")
         .getOrCreate())
```

On Databricks, **spark** already exists — don't create a new one.

Common configurations

Config	Default	Meaning
spark.sql.shuffle.partitions	200	Partitions after a shuffle (joins, aggregations)
spark.sql.adaptive.enabled	true (3.2+)	Adaptive Query Execution
spark.sql.autoBroadcastJoinThreshold	10 MB	Max table size to broadcast automatically
spark.sql.files.maxPartitionBytes	128 MB	Target size of input partitions when reading files
spark.executor.memory	varies	Heap per executor
spark.executor.cores	varies	Concurrent tasks per executor
spark.driver.memory	varies	Driver heap
spark.sql.sources.partitionOverwriteMode	static	**dynamic** = overwrite only written partitions
spark.sql.session.timeZone	JVM zone	Time zone for timestamp functions
spark.serializer	Java	**KryoSerializer** is faster for RDD work

```
spark.conf.set("spark.sql.shuffle.partitions", "400")   # runtime SQL configs
spark.conf.get("spark.sql.shuffle.partitions")
```

Executor and driver resources must be set before the application starts (spark-submit or cluster config), not at runtime.

7. Reading data

General form

```
df = (spark.read
      .format("csv")                    # csv, json, parquet, orc, avro, delta, jdbc, text
      .option("header", True)
      .schema(schema)                     # or .option("inferSchema", True)
      .load("/landing/orders/"))
```

Shortcuts: **spark.read.csv(path, header=True)**, **.json(path)**, **.parquet(path)**, **.table("db.table")**.

CSV

```
df = (spark.read
      .option("header", True)
      .option("sep", ",")
      .option("quote", "").option("escape", "")
      .option("multiLine", True)          # quoted fields with newlines
      .option("nullValue", "NA")
      .option("dateFormat", "yyyy-MM-dd")
      .option("timestampFormat", "yyyy-MM-dd HH:mm:ss")
      .option("mode", "PERMISSIVE")       # PERMISSIVE | DROPMALFORMED | FAILFAST
      .option("columnNameOfCorruptRecord", "_corrupt_record")
      .schema(schema)
      .csv("/landing/orders/*.csv"))
```

JSON

```
df = spark.read.json("/landing/events/")                        # one JSON object per line
df = spark.read.option("multiLine", True).json("/landing/api/")  # whole-file JSON arrays
```

Parquet / ORC / Avro / Delta

```
spark.read.parquet("/silver/orders")
spark.read.orc("/hive/orders")
spark.read.format("avro").load("/kafka_dump/")                    # needs spark-avro package
spark.read.format("delta").load("/delta/orders")                  # or spark.table("silver.orders")
spark.read.format("delta").option("versionAsOf", 12).load(path) # time travel
```

Schema inference: avoid in production

inferSchema=True reads the data an extra time and guesses types — IDs become integers and lose leading zeros, a column with one bad value becomes string. Give an explicit schema for CSV and JSON. Parquet, ORC, Avro and Delta store the schema in the files.

Corrupt records

Mode	Behaviour
PERMISSIVE (default)	Keeps the row, puts bad fields as null, raw line in the corrupt-record column (if in the schema)
DROPMALFORMED	Drops bad rows silently
FAILFAST	Fails the job on the first bad row

On Databricks, **.option("badRecordsPath", "/quarantine/orders")** writes bad records to files for review.

Reading many files

```
spark.read.parquet("/data/orders/order_date=2026-01-*")          # glob
spark.read.option("recursiveFileLookup", True).json("/raw/")    # all subfolders, no partition discovery
spark.read.option("pathGlobFilter", "*.json").json("/raw/")
spark.read.option("modifiedAfter", "2026-01-15T00:00:00").json("/raw/")
from pyspark.sql.functions import input_file_name
df.withColumn("source_file", input_file_name())                  # or _metadata.file_path in Databricks
```

Partition discovery

Folders named **key=value** become columns automatically:

```
/silver/orders/order_date=2026-01-14/part-0001.parquet
/silver/orders/order_date=2026-01-15/part-0001.parquet
```

spark.read.parquet("/silver/orders").filter("order_date = '2026-01-15'") reads only one folder — partition pruning.

JDBC sources

```
df = (spark.read.format("jdbc")
      .option("url", "jdbc:sqlserver://host:1433;databaseName=sales")
      .option("dbtable", "(SELECT * FROM dbo.orders WHERE order_date >= '2026-01-01') q")
      .option("user", user).option("password", pwd)
      .option("partitionColumn", "order_id")       # parallel read
      .option("lowerBound", 1).option("upperBound", 50_000_000)
      .option("numPartitions", 16)
      .option("fetchsize", 10_000)
      .load())
```

Without **partitionColumn**/bounds/**numPartitions**, Spark reads the whole table through **one** connection in one task. Bounds only split the range — they don't filter rows. Don't use more partitions than the source database can handle.

8. Writing data

General form

```
(df.write
    .format("parquet")              # parquet, delta, orc, csv, json, jdbc
    .mode("overwrite")              # append | overwrite | ignore | error (errorifexists)
    .partitionBy("order_date")
    .option("compression", "snappy")
    .save("/silver/orders"))

df.write.mode("append").saveAsTable("silver.orders")      # metastore table
```

Save modes

Mode	If target exists
error / errorifexists (default)	Fail
append	Add new files
overwrite	Replace the data (whole table, or partitions in dynamic mode)
ignore	Do nothing

Partition overwrite safely

```
spark.conf.set("spark.sql.sources.partitionOverwriteMode", "dynamic")
df_one_day.write.mode("overwrite").partitionBy("order_date").parquet("/silver/orders")
# only order_date partitions present in df_one_day are replaced

# Delta equivalent
(df_one_day.write.format("delta").mode("overwrite")
    .option("replaceWhere", "order_date = '2026-01-15'")
    .save("/delta/orders"))
```

Static overwrite on a partitioned table replaces the **entire** table with whatever you're writing. Losing all history because a backfill wrote one day with **mode("overwrite")** is one of the most common production accidents.

Controlling file count and size

Each task writes at least one file per partition value it holds. 200 shuffle partitions × 30 dates = up to 6,000 files.

```
df.repartition("order_date").write.partitionBy("order_date").parquet(path)   # ~1 file per date
df.coalesce(16).write.parquet(path)                                          # fewer files, no full shuffle
df.write.option("maxRecordsPerFile", 1_000_000).parquet(path)                # cap file size
```

Target files of roughly 128 MB–1 GB for analytics. On Delta, **OPTIMIZE** and auto-compaction handle this for you.

Bucketing

```
(df.write.bucketBy(64, "customer_id").sortBy("customer_id")
    .mode("overwrite").saveAsTable("silver.orders_bucketed"))
```

Two tables bucketed the same way on the join key can be joined without a shuffle. Works only with metastore tables (**saveAsTable**) and Spark's bucketing; less common now that Delta offers Z-order and liquid clustering.

Writing to JDBC

```
(df.write.format("jdbc").option("url", url).option("dbtable", "dbo.stage_orders")
    .option("user", user).option("password", pwd)
    .option("batchsize", 10_000).option("numPartitions", 8)
    .mode("append").save())
```

Write to a staging table, then MERGE inside the database, so failed runs don't leave partial data in the final table.

Commit safety

Plain Parquet writes are not atomic: a failed job can leave partial files and readers can see incomplete data. Delta Lake (or Iceberg/Hudi) adds a transaction log so writes are atomic.

Sagar

9. Schemas and data types

Defining schemas

```
from pyspark.sql.types import (StructType, StructField, StringType, IntegerType, LongType,
                               DecimalType, DateType, TimestampType, ArrayType, MapType,
BooleanType)

schema = StructType([
    StructField("order_id", LongType(), False),
    StructField("customer_id", LongType(), True),
    StructField("amount", DecimalType(18, 2), True),
    StructField("order_ts", TimestampType(), True),
    StructField("items", ArrayType(StructType([
        StructField("sku", StringType()), StructField("qty", IntegerType())
    ])), True),
    StructField("attrs", MapType(StringType(), StringType()), True),
])

# DDL string form — shorter
schema = "order_id BIGINT, customer_id BIGINT, amount DECIMAL(18,2), order_ts TIMESTAMP,
items ARRAY<STRUCT<sku: STRING, qty: INT>>"
```

Inspecting

```
df.printSchema()
df.schema                    # StructType
df.dtypes                    # [('order_id', 'bigint'), ...]
df.columns
df.schema.simpleString()
```

Casting

```
from pyspark.sql import functions as F
df = (df.withColumn("amount", F.col("amount").cast("decimal(18,2)"))
        .withColumn("order_date", F.to_date("order_ts"))
        .withColumn("customer_id", F.col("customer_id").cast("bigint")))
```

Invalid casts return **null** (unless ANSI mode is on, where they raise errors). After casting raw strings, count new nulls to catch bad data.

Nullable

nullable=False in a schema is a hint for the optimizer. When reading files, Spark generally does not enforce it. Enforce not-null rules with checks or Delta constraints.

Schema evolution

```
spark.read.option("mergeSchema", True).parquet(path)                # union of file schemas
df.write.format("delta").option("mergeSchema", True).mode("append").save(path)   # add new columns
df1.unionByName(df2, allowMissingColumns=True)                      # align by name, fill missing with null
```

Sagar

10. DataFrame transformations

Imports convention

```
from pyspark.sql import functions as F, Window as W
from pyspark.sql.types import *
```

Use **F.** for functions so they don't clash with Python built-ins like **sum**, **max**, **round**.

Selecting and renaming

```
df.select("order_id", "amount")
df.select(F.col("order_id"), (F.col("amount") * 1.18).alias("amount_with_tax"))
df.selectExpr("order_id", "amount * 1.18 AS amount_with_tax", "upper(status) AS status")
df.withColumnRenamed("amt", "amount")
df.toDF(*[c.lower() for c in df.columns])          # rename all
df.drop("tmp1", "tmp2")
```

Adding and changing columns

```
df = (df.withColumn("is_big", F.col("amount") > 1000)
        .withColumn("load_ts", F.current_timestamp())
        .withColumn("source", F.lit("pos_system")))

# many at once (Spark 3.3+)
df = df.withColumns({"is_big": F.col("amount") > 1000, "source": F.lit("pos")})
```

> Calling **withColumn** in a loop for hundreds of columns creates a deep plan that is slow to analyze. Build one **select** with a list of expressions instead.

```
df = df.select([F.trim(F.col(c)).alias(c) if t == "string" else F.col(c) for c, t in df.dtypes])
```

Filtering

```
df.filter(F.col("amount") > 1000)
df.where("amount > 1000 AND status = 'PAID'")
df.filter((F.col("status") == "PAID") & (F.col("city").isin("Pune", "Bengaluru")))
df.filter(F.col("email").isNull())
df.filter(F.col("name").rlike("^[A-Z]"))
df.filter(~F.col("status").isin("CANCELLED", "RETURNED"))
```

Use **&**, **|**, **~** — not **and**, **or**, **not** — and wrap each condition in parentheses.

Conditional logic

```
df = df.withColumn("band",
        F.when(F.col("amount") >= 10_000, "high")
         .when(F.col("amount") >= 1_000, "medium")
         .otherwise("low"))
```

Sagar

Sorting, distinct, limits

```
df.orderBy(F.col("amount").desc(), "order_id")
df.orderBy(F.desc_nulls_last("amount"))
df.sortWithinPartitions("order_ts")          # no global sort, no full shuffle
df.distinct()
df.dropDuplicates(["order_id"])               # keeps an arbitrary row per key
df.limit(100)
```

Unions

```
df1.union(df2)                                # by position — dangerous
df1.unionByName(df2)                          # by name — prefer this
df1.unionByName(df2, allowMissingColumns=True)
```

Looking at data (actions)

```
df.show(20, truncate=False); df.show(vertical=True)
df.count()
df.take(5); df.first(); df.head(5)
df.collect()                  # all rows to the driver — only for small results
df.toPandas()                 # same warning
df.describe().show(); df.summary().show()
```

Sagar

II. Built-in functions

*Always check **pyspark.sql.functions** before writing a UDF. There are several hundred built-in functions and they all run inside the JVM, optimized.*

Strings

```
F.upper, F.lower, F.initcap, F.trim, F.ltrim, F.rtrim, F.length
F.substring("s", 1, 3), F.concat("a", "b"), F.concat_ws("-", "a", "b")
F.split("tags", ","), F.regexp_extract("email", r"@(.+)$", 1), F.regexp_replace("phone", r"[^0-9]", "")
F.lpad("id", 8, "0"), F.translate("s", "abc", "xyz"), F.instr("s", "x")
F.format_string("%s-%05d", "region", "id"), F.sha2(F.concat_ws("|", *cols), 256), F.md5("s")
```

Numbers

```
F.round("amount", 2), F.bround, F.ceil, F.floor, F.abs, F.pow, F.sqrt, F.log
F.greatest("a", "b", "c"), F.least(...)
F.rand(seed=42), F.monotonically_increasing_id()   # unique but not consecutive
```

Dates and timestamps

```
F.current_date(), F.current_timestamp()
F.to_date("s", "dd-MM-yyyy"), F.to_timestamp("s", "yyyy-MM-dd HH:mm:ss")
F.date_format("ts", "yyyy-MM"), F.year, F.month, F.dayofmonth, F.dayofweek, F.hour, F.weekofyear
F.date_add("d", 7), F.date_sub("d", 7), F.add_months("d", 1)
F.datediff("end", "start"), F.months_between("end", "start")
F.trunc("d", "month"), F.date_trunc("hour", "ts"), F.last_day("d"), F.next_day("d", "Mon")
F.unix_timestamp("ts"), F.from_unixtime("epoch"), F.timestamp_seconds("epoch")
F.from_utc_timestamp("ts", "Asia/Kolkata"), F.to_utc_timestamp("ts", "Asia/Kolkata")
F.sequence(F.lit("2026-01-01").cast("date"), F.lit("2026-01-31").cast("date"))
```

NULL handling

```
F.coalesce("phone", "mobile", F.lit("N/A"))
F.when(F.col("x").isNull(), 0).otherwise(F.col("x"))
F.nanvl("x", F.lit(0))                    # NaN (not null) replacement for doubles
df.fillna({"city": "UNKNOWN", "amount": 0})
df.dropna(subset=["order_id"])
df.na.replace({"N/A": None}, subset=["phone"])
F.col("a").eqNullSafe(F.col("b"))         # NULL-safe equality (<=> in SQL)
```

Arrays, maps and structs

```
F.array("a", "b"), F.size("items"), F.array_contains("tags", "vip")
F.element_at("items", 1), F.array_distinct, F.array_union, F.array_except, F.sort_array
F.explode("items"), F.explode_outer("items"), F.posexplode("items")
F.flatten("nested"), F.arrays_zip("a", "b")
F.struct("city", "zip").alias("address"), F.col("address.city")
F.create_map(F.lit("k"), F.col("v")), F.map_keys("m"), F.map_values("m"), F.col("m")["key"]
F.collect_list("x"), F.collect_set("x")
```

Higher-order functions (run on array elements without exploding)

```
F.transform("items", lambda x: x["qty"] * 2)
F.filter("items", lambda x: x["qty"] > 0)
F.exists("items", lambda x: x["sku"] == "A1")
F.aggregate("items", F.lit(0), lambda acc, x: acc + x["qty"])
```

JSON

```
F.from_json("payload", schema)              # string → struct
F.to_json(F.struct("*"))                    # columns → JSON string
F.get_json_object("payload", "$.customer.id")
F.json_tuple("payload", "id", "city")
F.schema_of_json(F.lit(sample_json))
```

Flattening a nested struct generically

```
def flatten(df):
    while True:
        structs = [f.name for f in df.schema.fields if isinstance(f.dataType, StructType)]
        if not structs:
            return df
        cols = [c for c in df.columns if c not in structs]
        for s in structs:
            cols += [F.col(f"{s}.{n}").alias(f"{s}_{n}") for n in df.select(f"{s}.*").columns]
        df = df.select(cols)
```

Sagar

12. Aggregations

groupBy + agg

```
summary = (df.groupBy("order_date", "city")
              .agg(F.count("*").alias("orders"),
                   F.countDistinct("customer_id").alias("customers"),
                   F.sum("amount").alias("revenue"),
                   F.avg("amount").alias("avg_order"),
                   F.max("order_ts").alias("last_order"),
                   F.sum(F.when(F.col("status") == "CANCELLED",
1).otherwise(0)).alias("cancelled")))
```

Approximate and percentile

```
F.approx_count_distinct("user_id", rsd=0.02)
F.percentile_approx("amount", 0.5)                  # median
F.percentile_approx("amount", [0.25, 0.5, 0.75])
```

How Spark aggregates

Spark does a **partial aggregation** on each partition before the shuffle (like a combiner), then a final aggregation after it. Sums and counts shrink massively before data moves. **collect_list** and **countDistinct** shrink less, so they shuffle more.

Pivot

```
df.groupBy("product").pivot("month", ["Jan", "Feb", "Mar"]).sum("amount")
```

Always pass the list of pivot values — otherwise Spark runs an extra job to find them, and column order isn't stable.

Unpivot

```
df.unpivot(["product"], ["jan", "feb", "mar"], "month", "amount")      # Spark 3.4+
df.selectExpr("product", "stack(3, 'jan', jan, 'feb', feb, 'mar', mar) AS (month, amount)")
```

Rollup and cube

```
df.rollup("region", "city").agg(F.sum("amount"))
df.cube("region", "channel").agg(F.sum("amount"))
F.grouping("city")          # 1 on subtotal rows
```

Sagar

13. Joins

Syntax

```
orders.join(customers, "customer_id")                          # inner, single shared column
orders.join(customers, ["customer_id", "region"], "left")
orders.join(customers, orders.customer_id == customers.id, "left")
o, c = orders.alias("o"), customers.alias("c")
o.join(c, F.col("o.customer_id") == F.col("c.id"), "inner").select("o.*", "c.name")
```

Join types: **inner**, **left/left_outer**, **right**, **full/outer**, **left_semi**, **left_anti**, **cross**.

Ambiguous columns

Joining with an expression keeps both key columns. Use a column-name join (**"customer_id"**) to get one, or alias the DataFrames and select explicitly. Self joins always need aliases.

Join strategies

In simple words: to join two tables, matching keys must meet on the same machine. Either **send the small table to every machine** (broadcast join — cheap, no shuffle), or **shuffle both tables by key** so matching rows land together (sort-merge join — works for any size, but expensive). Picking the right one is most of join tuning.

Memory hook: "Small table? Spread it. Both big? Sort and merge." — broadcast the small one to everyone; otherwise both sides shuffle, sort and meet.

Strategy	When Spark uses it	Shuffle?
Broadcast hash join	One side is small (below threshold or hinted)	No — small table is copied to every executor
Sort-merge join	Default for two large tables, equi-join	Yes — both sides shuffled and sorted
Shuffle hash join	One side much smaller but too big to broadcast; enabled by config/AQE	Yes, but no sort
Broadcast nested loop	Non-equi join with a small side	No, but expensive
Cartesian	Cross join, non-equi with two big sides	Very expensive

Broadcast joins

```
from pyspark.sql.functions import broadcast
fact.join(broadcast(dim), "product_id")
```

- The broadcast table goes to the driver first, then to every executor. Broadcasting a big table causes driver OOM.
- Default auto threshold: 10 MB (**spark.sql.autoBroadcastJoinThreshold**, size estimated from statistics).

- With AQE, Spark can switch to a broadcast join at runtime once it sees the actual size.

Join hints

```
df1.join(df2.hint("broadcast"), "id")
df1.join(df2.hint("merge"), "id")          # sort-merge
df1.join(df2.hint("shuffle_hash"), "id")
```

Skewed joins

If a few keys hold most rows, the tasks for those keys run far longer than the rest — the stage waits on them.

Fixes, in order:

1. Enable AQE skew join handling (**spark.sql.adaptive.skewJoin.enabled=true**) — splits large partitions automatically for sort-merge joins.
2. Broadcast the other side if it's small enough.
3. Handle the hot keys separately (join them with broadcast, union with the rest).
4. **Salting:** spread each hot key over N sub-keys.

```
N = 16
big_s = big.withColumn("salt", (F.rand() * N).cast("int"))
small_s = small.withColumn("salt", F.explode(F.sequence(F.lit(0), F.lit(N - 1))))
joined = big_s.join(small_s, ["customer_id", "salt"]).drop("salt")
```

NULL keys

NULLs never match in equality joins. Many NULL keys on a left join all land in one shuffle partition — a hidden skew. Filter or handle NULL keys separately.

Fan-out

Duplicate keys on both sides multiply rows. Check uniqueness on the side that should be unique before joining:

```
dim.groupBy("product_id").count().filter("count > 1").count()   # should be 0
```

14. Window functions

In simple words: ***groupBy*** *squashes many rows into one per group. A window function keeps every row and adds a value computed from its "neighbours" in the same group — its rank, the previous row's value, a running total. "For each customer, ordered by date, give me..." is almost always a window function.*

Memory hook: *groupBy is a blender (many rows become one smoothie); a window is a class photo (everyone stays in the picture, each gets a label like rank or running total).*

Defining windows

```
w = W.partitionBy("customer_id").orderBy("order_ts")
w_running = w.rowsBetween(W.unboundedPreceding, W.currentRow)
w_7 = w.rowsBetween(-6, 0)
w_all = W.partitionBy("customer_id")
```

Common uses

```
df = (df
  .withColumn("rn", F.row_number().over(w))
  .withColumn("rank", F.rank().over(W.partitionBy("dept").orderBy(F.desc("salary"))))
  .withColumn("prev_amount", F.lag("amount").over(w))
  .withColumn("next_amount", F.lead("amount").over(w))
  .withColumn("running_total", F.sum("amount").over(w_running))
  .withColumn("avg_7", F.avg("amount").over(w_7))
  .withColumn("customer_total", F.sum("amount").over(w_all))
  .withColumn("first_order", F.first("order_ts").over(w_running))
  .withColumn("filled", F.last("temp", ignorenulls=True).over(w_running))
  .withColumn("quartile", F.ntile(4).over(W.orderBy(F.desc("amount")))))
```

Latest record per key

```
w = W.partitionBy("order_id").orderBy(F.desc("updated_at"), F.desc("ingest_ts"))
latest = df.withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")
```

Range-based frames

```
w_days = (W.partitionBy("store_id")
          .orderBy(F.col("sale_date").cast("timestamp").cast("long"))
          .rangeBetween(-6 * 86400, 0))          # last 7 days by value, not rows
df.withColumn("sales_7d", F.sum("amount").over(w_days))
```

Performance

- *Every window needs its data partitioned and sorted by its spec — a shuffle and a sort.*
- *A window without* ***partitionBy*** *puts* ***all*** *data in one partition on one executor. Spark warns about it. Only acceptable on small, already-aggregated data.*

- Skewed partition keys create one huge task.

sagar

15. Spark SQL

Temporary views

```
df.createOrReplaceTempView("orders")                    # session scope
spark.sql("""
    SELECT customer_id, SUM(amount) AS total
    FROM orders
    WHERE order_date >= '2026-01-01'
    GROUP BY customer_id
""").show()

df.createOrReplaceGlobalTempView("orders_g")          # shared across sessions:
global_temp.orders_g
```

DataFrame code and SQL compile to the same plan — choose whichever is clearer. SQL is often clearer for complex joins and windows; DataFrames are better for dynamic, programmatic logic.

Catalog and tables

```
spark.sql("CREATE DATABASE IF NOT EXISTS silver")
spark.sql("USE silver")
spark.catalog.listTables("silver")
spark.table("silver.orders")
spark.sql("DESCRIBE EXTENDED silver.orders")
spark.sql("SHOW PARTITIONS silver.orders")
spark.sql("ANALYZE TABLE silver.orders COMPUTE STATISTICS FOR ALL COLUMNS")
```

Parameterized SQL

```
spark.sql("SELECT * FROM orders WHERE order_date = :d", args={"d": "2026-01-15"})   # Spark 3.4+
```

Avoid building SQL with f-strings from user input.

Sagar

16. UDFs and pandas integration

Python UDFs: last resort

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

@udf(returnType=StringType())
def mask_email(e):
    if e is None: return None
    user, _, dom = e.partition("@")
    return user[:2] + "***@" + dom

df.withColumn("email_masked", mask_email("email"))
```

Why they're slow:

- Each row is serialized from the JVM to a Python worker process and back.
- Catalyst can't see inside them, so no optimizations (no predicate pushdown through them).

The same logic with built-ins:

```
df.withColumn("email_masked",
    F.concat(F.substring("email", 1, 2), F.lit("***@"), F.substring_index("email", "@", -1)))
```

pandas UDFs (vectorized)

```
import pandas as pd
from pyspark.sql.functions import pandas_udf

@pandas_udf("double")
def to_inr(amount: pd.Series, rate: pd.Series) -> pd.Series:
    return amount * rate

df.withColumn("amount_inr", to_inr("amount", "fx_rate"))
```

Data moves in Arrow batches, and the function runs on whole pandas Series — often 10× or more faster than a row UDF.

Grouped processing with applyInPandas

```
def zscore(pdf: pd.DataFrame) -> pd.DataFrame:
    pdf["z"] = (pdf["amount"] - pdf["amount"].mean()) / pdf["amount"].std()
    return pdf

out = df.groupBy("store_id").applyInPandas(zscore, schema="store_id long, amount double, z
double")
```

Each group must fit in one executor's memory.

Other bridges

```
df.mapInPandas(fn, schema)                          # iterate pandas batches per partition
pdf = df.limit(1000).toPandas()                     # small results only
sdf = spark.createDataFrame(pdf)                    # pandas → Spark
spark.conf.set("spark.sql.execution.arrow.pyspark.enabled", "true")   # faster
toPandas/createDataFrame
```

Choosing

1. Built-in functions (always first)
2. Spark SQL expressions / higher-order functions
3. pandas UDF / applyInPandas
4. Python UDF

Sagar

17. Partitioning

In simple words: *partitioning is about how data is split — when it's read (input partitions), after a shuffle (shuffle partitions), and how it's stored in folders (storage partitions). Get the sizes right (~100–200 MB per partition) and Spark uses every core without running out of memory.*

Memory hook: *Repartition = Reshuffle (both start with R — full shuffle, can go up or down). Coalesce = Combine (both start with C — merge neighbours, only down).*

Three kinds of "partitioning"

Kind	What	Controlled by
Input partitions	How files are split into tasks when reading	**spark.sql.files.maxPartitionBytes** (128 MB), file count and sizes
Shuffle partitions	Number of tasks after a shuffle	**spark.sql.shuffle.partitions** (200), AQE coalescing
Storage partitions	Folder layout on disk (**partitionBy**)	Your write code

Checking partitions

```
df.rdd.getNumPartitions()
df.groupBy(F.spark_partition_id()).count().show()     # rows per partition — spot skew
```

repartition vs coalesce

	repartition(n) / repartition("col")	coalesce(n)
Can increase partitions	Yes	No
Shuffle	Full shuffle	No shuffle (merges neighbours)
Result	Even sizes; by-column puts each key in one partition	Can be uneven
Use for	Fix skew/parallelism, prepare for **partitionBy** writes, joins	Reduce file count before writing

coalesce(1) *before writing forces the entire upstream stage onto one task in many plans. Use it only for tiny outputs.*

Sizing shuffle partitions

Aim for ~100–200 MB per shuffle partition. For 500 GB of shuffle data that's roughly 2,500–5,000 partitions; for 2 GB, the default 200 is too many. With AQE enabled, set a reasonably high initial number and let AQE coalesce small partitions.

Sagar

Partition pruning and predicate pushdown

- **Partition pruning:** filters on storage-partition columns skip whole folders.
- **Predicate pushdown:** filters on other columns are pushed to the file reader; Parquet/ORC skip row groups using min/max statistics.
- **Column pruning:** only selected columns are read from columnar files.

```
df = spark.read.parquet("/silver/orders")
df.filter("order_date = '2026-01-15' AND amount > 1000").select("order_id", "amount").explain()
# PartitionFilters: [order_date = 2026-01-15]   PushedFilters: [GreaterThan(amount,1000)]
```

Functions on the partition column (**year(order_date) = 2026**) can block pruning — filter the raw column with a range.

Choosing a storage partition column

- Used in most filters (usually a date).
- Low to medium cardinality — thousands of partitions at most, not millions.
- Each partition should hold enough data for decent file sizes. Partitioning a 10 GB table by day for 10 years gives ~3 MB per day — too small; partition by month or not at all, and use clustering instead.

Sagar

18. Caching and persistence

When to cache

In simple words: *Spark forgets results after each action and recomputes from the start next time. Caching says "keep this result in memory because I'll use it again". Useful when reused; wasteful when used once.*

Memory hook: *the* ***fridge*** *— put food in only if you'll eat it again soon, and clear it out after (**unpersist**). Cache is the Common default; Persist lets you Pick the level.*

Cache a DataFrame when it is ***reused by multiple actions*** *and is expensive to recompute — for example a cleaned dataset that feeds three different aggregations.*

```
clean = raw.filter(...).dropDuplicates(["id"]).cache()
clean.count()                          # materializes the cache
clean.groupBy("city").count().write...
clean.groupBy("status").count().write...
clean.unpersist()
```

Storage levels

Level	Where	Notes
MEMORY_ONLY	Memory, deserialized	Fastest; partitions that don't fit are recomputed
MEMORY_AND_DISK	Memory, spill to disk	**df.cache()** default for DataFrames
MEMORY_ONLY_SER / MEMORY_AND_DISK_SER	Serialized	Less memory, more CPU (RDD API)
DISK_ONLY	Disk	When memory is tight
OFF_HEAP	Off-heap memory	Needs config

```
from pyspark import StorageLevel
df.persist(StorageLevel.MEMORY_AND_DISK)
```

Don't cache by default

- Caching costs memory that execution needs; over-caching causes spills and OOMs.
- A DataFrame used once gains nothing from caching.
- Caching is lazy — it fills on the first action.
- Always **unpersist()** when done in long jobs.
- On Databricks, the disk cache speeds up repeated reads of the same files automatically.

Sagar

Checkpointing

```
spark.sparkContext.setCheckpointDir("/tmp/checkpoints")
df = df.checkpoint()          # writes data, truncates lineage
df = df.localCheckpoint()     # executor-local, faster, less safe
```

Use checkpointing for very long lineages (iterative loops) where the plan itself becomes slow to optimize, or recomputation after failure would be huge.

Sagar

19. Catalyst, Tungsten and Adaptive Query Execution

In simple words: Catalyst rewrites your query into a faster equivalent before it runs (like a GPS finding a better route). **Tungsten** makes the engine itself efficient (compact memory, generated code). **AQE** changes the route while driving, using real traffic data from the parts of the job already finished.

Memory hook — the three helpers: Catalyst = the planner (before the trip), **Tungsten** = the engine (how fast the car runs), **AQE** = live traffic updates (during the trip).

Catalyst optimizer

Every DataFrame or SQL query goes through:

1. **Unresolved logical plan** — parsed, names not checked.
2. **Analyzed logical plan** — columns and tables resolved against the catalog.
3. **Optimized logical plan** — rule-based rewrites: predicate pushdown, column pruning, constant folding, combining filters, simplifying expressions.
4. **Physical plans** — candidate execution strategies (e.g. broadcast vs sort-merge join), chosen with cost estimates.
5. **Code generation** — whole-stage codegen compiles parts of the plan into Java bytecode.

```
df.explain()                    # physical plan
df.explain(mode="extended")     # all plans
df.explain(mode="formatted")    # readable sections
df.explain(mode="cost")         # with statistics
```

What to look for in a plan:

- **Exchange** = shuffle. Count them.
- **BroadcastHashJoin** vs **SortMergeJoin**.
- **PushedFilters**, **PartitionFilters** in the scan.
- ***(1)** stars = whole-stage codegen.
- **BatchEvalPython** / **ArrowEvalPython** = Python UDFs.

Tungsten

Tungsten is Spark's execution layer: compact binary row format, off-heap memory management, cache-aware algorithms and generated code. It's why DataFrames are much faster than RDDs of Python objects.

Adaptive Query Execution (AQE)

AQE re-optimizes the plan **during** execution using real statistics collected at shuffle boundaries. Enabled by default since Spark 3.2.

Feature	What it does	Config
Coalesce shuffle partitions	Merges small post-shuffle partitions into fewer, right-sized ones	**spark.sql.adaptive.coalescePartitions.enabled**
Switch join strategy	Converts sort-merge to broadcast when a side turns out small	automatic
Skew join	Splits oversized partitions in sort-merge joins	**spark.sql.adaptive.skewJoin.enabled**
Local shuffle reader	Avoids network reads after converting to broadcast	automatic

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.advisoryPartitionSizeInBytes", "128m")
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionFactor", "5")
```

Dynamic Partition Pruning (DPP)

When a partitioned fact table is joined to a filtered dimension, Spark uses the dimension's filter result at runtime to prune fact partitions:

```
SELECT ... FROM sales s JOIN dim_date d ON s.sale_date = d.date
WHERE d.fiscal_quarter = 'FY26-Q1'      -- only matching sale_date partitions are read
```

Statistics

The optimizer's choices depend on size and cardinality estimates. Keep table statistics fresh (**ANALYZE TABLE ... COMPUTE STATISTICS**), especially for broadcast decisions without AQE.

20. Memory and cluster sizing

In simple words: *each executor has a fixed memory budget. Part of it is for running tasks (shuffles, joins, sorts), part for cached data, and some extra outside the JVM for Python and overheads. Most "out of memory" errors mean one task got too much data — usually because of too few partitions or skew — not that the cluster is too small.*

Memory hook: *"OOM on the driver = too much came to the boss (collect, big broadcast). OOM on an executor = one basket was too heavy (skew, too few partitions)."*

Executor memory layout

```
spark.executor.memory  (JVM heap)
├── Reserved memory              ~300 MB
├── User memory                  (1 - spark.memory.fraction)  ~40%: your objects,
UDF data structures
└── Unified memory               spark.memory.fraction = 0.6
    ├── Execution memory         shuffles, joins, sorts, aggregations
    └── Storage memory           cache, broadcast variables
        (spark.memory.storageFraction = 0.5 is protected from eviction; the
boundary moves)
+ spark.executor.memoryOverhead  off-heap: JVM overhead, Python workers, network
buffers (default max(384MB, 10%))
```

Execution can borrow from storage (evicting cached blocks) and vice versa within the unified region.

Sizing executors

Common starting point on YARN/Kubernetes:

- **4–5 cores per executor** *(more cores per JVM increases GC and I/O contention).*
- *Memory per executor = (node memory − OS overhead) / executors per node, minus memoryOverhead.*
- *Leave a core and some memory per node for the OS and node manager.*
- *PySpark with UDFs/pandas UDFs needs more* **memoryOverhead**, *because Python workers live outside the JVM heap.*

On Databricks you mostly pick node types and worker counts; Spark configs are derived for you.

Dynamic allocation

```
spark.dynamicAllocation.enabled=true
spark.dynamicAllocation.minExecutors=2
spark.dynamicAllocation.maxExecutors=50
```

Executors are added when tasks are pending and removed when idle. Needs shuffle tracking or an external shuffle service.

Out of memory: driver vs executor

Where	Typical causes	Fixes
Driver	**collect()/toPandas()** on big data, broadcasting large tables, huge numbers of small files/partitions to plan, very large plans	Write instead of collect, don't broadcast big tables, compact files, increase driver memory last
Executor	Skewed partitions, too few shuffle partitions (huge partitions), big **collect_list**/explode, large Python UDF batches, over-caching	More/even partitions, fix skew, AQE, reduce cached data, raise memoryOverhead for Python

Spill

When execution memory runs out, Spark spills sorted/hashed data to disk. Some spill is normal; heavy spill (visible in the Spark UI as "Spill (Memory)" and "Spill (Disk)") means partitions are too big — increase partition count or memory.

Garbage collection

Long GC times (shown per task in the Spark UI) point to memory pressure: too many cores sharing a heap, very large objects, or too much cached data. Use fewer cores per executor, smaller partitions, or serialized caching.

Sagar

21. Performance playbook

Read less

- Columnar formats (Parquet/Delta), with compression.
- Select only needed columns early.
- Filter early, on partition columns where possible.
- Partition and cluster tables for the common filters.

Shuffle less

- Broadcast small dimension tables.
- Aggregate before joining when it reduces rows.
- Avoid unnecessary **distinct**, **orderBy**, **repartition**.
- Reuse the same partitioning for consecutive operations on the same key.
- Bucket or cluster large tables joined repeatedly on the same key.

Balance work

- Right-size shuffle partitions (AQE helps).
- Detect skew (Spark UI: max task time far above median) and fix it: AQE skew join, broadcast, isolate hot keys, salting.
- Avoid windows without **partitionBy**.

Avoid Python overhead

- Built-in functions over UDFs; pandas UDFs over Python UDFs.
- No **collect()** in loops; no per-row Python logic on the driver.

Keep plans healthy

- No **withColumn** in long loops — use one **select**.
- Checkpoint very long iterative lineages.
- Avoid repeated **count()** calls used only for logging on huge DataFrames; each is a full job.

Fix the small-file problem

Thousands of tiny files slow down listing, planning and reading.

- Repartition by the partition column before writing.
- Compact with Delta **OPTIMIZE**, auto-compaction or optimized writes.
- Batch streaming writes with sensible triggers.

Sagar

Common anti-patterns

Anti-pattern	Better
`df.collect()` then loop in Python	Express the logic with DataFrame operations
`inferSchema` on large CSV in production	Explicit schema
`coalesce(1)` for big outputs	Repartition to sensible file sizes
Python UDF for string cleaning	`regexp_replace`, `trim`, `lower`
`union` of many small DataFrames in a loop	Read all files in one `spark.read`
Joining on mismatched types	Cast once, join on matching types
Caching everything	Cache only reused, expensive DataFrames

Sagar

22. Reading the Spark UI

Tabs

Tab	Shows
Jobs	One row per action; duration, stages, status
Stages	Tasks per stage, input/output, shuffle read/write, spill; task-time distribution
Storage	Cached DataFrames, size in memory/disk
Environment	All Spark configs actually in effect
Executors	Memory, cores, task counts, GC time, shuffle per executor; failed executors
SQL / DataFrame	Query plans with per-operator metrics (rows, time, size)
Structured Streaming	Input rate, processing rate, batch duration, state size

Diagnosing from the UI

Symptom	Likely cause
One or a few tasks take much longer than the median	Skew
Large "Spill (Disk)"	Partitions too big for memory
Thousands of tasks of a few KB each	Too many small partitions/files
Few tasks while many cores sit idle	Too few partitions
High GC time	Memory pressure
Huge shuffle write	A wide transformation you might avoid (broadcast, pre-aggregate)
Long gap before tasks start	File listing, driver-side planning, many small files
Executors lost / tasks retried	OOM or node failure — check executor logs

Compare a fast run and a slow run of the same job side by side: input size, number of tasks, shuffle sizes and max task time usually point straight at the cause.

23. Structured Streaming

The model

In simple words: *Structured Streaming runs your normal DataFrame code again and again on small batches of new data (every few seconds or minutes), remembering where it stopped in a* ***checkpoint****. You write it almost like batch code; Spark handles the "only process new data" part.*

Memory hook — exactly-once is "R·C·I", Really Careful Ingestion: *Replayable source + Checkpoint + Idempotent sink. Miss one and you get duplicates or gaps.*

A stream is treated as an ***unbounded table****. Each trigger, new rows are appended to the input table, your query runs incrementally on them, and results are written to the sink. You use the same DataFrame API as batch.*

Sources and sinks

Sources	Sinks
Kafka, Event Hubs (Kafka API), Kinesis	Delta, Parquet/files
Files arriving in a folder (Auto Loader on Databricks)	Kafka
Delta tables (as a stream)	**foreachBatch** (any batch writer: JDBC, MERGE)
Rate source (testing)	Console, memory (testing)

A complete Kafka → Delta stream

```
schema = "event_id STRING, user_id STRING, event_ts TIMESTAMP, action STRING"

raw = (spark.readStream.format("kafka")
       .option("kafka.bootstrap.servers", "broker:9092")
       .option("subscribe", "clicks")
       .option("startingOffsets", "latest")
       .option("maxOffsetsPerTrigger", 500_000)
       .load())

events = (raw.select(F.from_json(F.col("value").cast("string"), schema).alias("e"))
             .select("e.*"))

q = (events.writeStream
     .format("delta")
     .outputMode("append")
     .option("checkpointLocation", "/chk/clicks_bronze")
     .trigger(processingTime="1 minute")
     .toTable("bronze.clicks"))

q.awaitTermination()
```

Triggers

Trigger	Behaviour
default (none)	Next micro-batch starts as soon as the previous finishes
processingTime="1 minute"	Micro-batch every interval
availableNow=True	Process everything available now in batches, then stop — great for scheduled "incremental batch" jobs
once=True	Older single-batch version of availableNow
continuous="1 second"	Experimental low-latency mode, limited operations

Output modes

Mode	Emits	Use with
append	Only new final rows	Stateless queries; windowed aggregations with a watermark
update	Rows changed in this batch	Aggregations where the sink supports upserts
complete	The whole result table each batch	Small aggregations only

Checkpoints

The checkpoint location stores source offsets, state and commit logs. It is what lets a stream restart exactly where it stopped.

- One unique checkpoint per query; never share between queries.
- Don't delete it casually — the stream will reprocess from **startingOffsets**.
- Some query changes (different aggregations, changed stateful operators) are incompatible with an existing checkpoint.

Event time, watermarks and windows

```
counts = (events
          .withWatermark("event_ts", "15 minutes")
          .groupBy(F.window("event_ts", "10 minutes"), "action")
          .count())
```

- **Event time:** when the event happened (inside the data). **Processing time:** when Spark sees it.
- **Watermark:** "I don't expect events older than max(event_ts) − 15 minutes". Spark keeps state for windows only until the watermark passes them, then finalizes and drops the state. Events later than the watermark are dropped.
- Window types: tumbling **window(ts, "10 minutes")**, sliding **window(ts, "10 minutes", "5 minutes")**, session **session_window(ts, "30 minutes")**.

Stateful operations

Aggregations, stream-stream joins, deduplication and **applyInPandasWithState** keep state across batches. Without a watermark, state grows forever. Use the RocksDB state store for large state:

```
spark.conf.set("spark.sql.streaming.stateStore.providerClass",
               "org.apache.spark.sql.execution.streaming.state.RocksDBStateStoreProvider")
```

Deduplication

```
deduped = (events.withWatermark("event_ts", "30 minutes")
                 .dropDuplicatesWithinWatermark(["event_id"]))       # Spark 3.5+
# older: .dropDuplicates(["event_id", "event_ts"]) with a watermark
```

Stream-static and stream-stream joins

```
enriched = events.join(broadcast(dim_users), "user_id", "left")       # stream-static: dim re-read
per batch (Delta)

joined = (impressions.withWatermark("imp_ts", "1 hour")
          .join(clicks.withWatermark("click_ts", "2 hours"),
                F.expr("""click_ad_id = imp_ad_id AND
                          click_ts BETWEEN imp_ts AND imp_ts + interval 1 hour""")))
```

Stream-stream joins need watermarks on both sides and a time-range condition so state can be cleaned up.

foreachBatch: run batch logic per micro-batch

```
from delta.tables import DeltaTable

def upsert(batch_df, batch_id):
    latest = (batch_df.withColumn("rn", F.row_number().over(
                  W.partitionBy("order_id").orderBy(F.desc("updated_at"))))
              .filter("rn = 1").drop("rn"))
    (DeltaTable.forName(spark, "silver.orders").alias("t")
       .merge(latest.alias("s"), "t.order_id = s.order_id")
       .whenMatchedUpdateAll()
       .whenNotMatchedInsertAll()
       .execute())

(orders_stream.writeStream
   .foreachBatch(upsert)
   .option("checkpointLocation", "/chk/orders_silver")
   .trigger(availableNow=True)
   .start())
```

foreachBatch lets you use MERGE, JDBC writes, or write one batch to several sinks.

Exactly-once

End-to-end exactly-once needs all three:

1. A **replayable source** (Kafka offsets, files, Delta).
2. **Checkpointed** progress.
3. An **idempotent or transactional sink** (Delta, or MERGE by key in **foreachBatch**; use **batch_id** to skip already-committed batches when writing elsewhere).

Kafka sinks are at-least-once — downstream consumers should deduplicate.

Monitoring and operations

```
q.status; q.lastProgress; q.recentProgress
spark.streams.active
q.stop()
```

Watch input rate vs processing rate (falling behind?), batch duration vs trigger interval, and state size. Alert when a query stops or lags.

Sagar

24. Testing and data quality

Local SparkSession for tests

```
# tests/conftest.py
import pytest
from pyspark.sql import SparkSession

@pytest.fixture(scope="session")
def spark():
    s = (SparkSession.builder.master("local[2]").appName("tests")
         .config("spark.sql.shuffle.partitions", "2").getOrCreate())
    yield s
    s.stop()
```

Testing a transformation

```
from pyspark.testing import assertDataFrameEqual      # Spark 3.5+

def test_latest_per_key(spark):
    df = spark.createDataFrame(
        [(1, "A", "2026-01-01"), (1, "B", "2026-01-02"), (2, "C", "2026-01-01")],
        "id INT, city STRING, updated_at STRING")
    expected = spark.createDataFrame([(1, "B"), (2, "C")], "id INT, city STRING")
    assertDataFrameEqual(latest_per_key(df).select("id", "city"), expected)
```

Before 3.5, compare **sorted(df.collect())** lists, or use the **chispa** library.

In-pipeline checks

```
def check(df, name):
    total = df.count()
    dupes = df.groupBy("order_id").count().filter("count > 1").count()
    nulls = df.filter(F.col("order_id").isNull()).count()
    if dupes or nulls:
        raise ValueError(f"{name}: {dupes} duplicate keys, {nulls} null keys out of {total}")
```

Compute several metrics in one aggregation to avoid many separate jobs:

```
m = df.agg(F.count("*").alias("rows"),
           F.countDistinct("order_id").alias("keys"),
           F.sum(F.col("order_id").isNull().cast("int")).alias("null_keys")).first()
```

Libraries: Great Expectations, Deequ / PyDeequ, Soda, and Delta Live Tables expectations on Databricks.

25. Common pipeline patterns

Bronze → silver cleaning

```
silver = (spark.read.table("bronze.orders")
  .withColumn("order_id", F.col("order_id").cast("bigint"))
  .withColumn("amount", F.col("amount").cast("decimal(18,2)"))
  .withColumn("order_ts", F.to_timestamp("order_ts"))
  .withColumn("order_date", F.to_date("order_ts"))
  .withColumn("email", F.lower(F.trim("email")))
  .filter(F.col("order_id").isNotNull())
  .withColumn("rn", F.row_number().over(
      W.partitionBy("order_id").orderBy(F.desc("updated_at"), F.desc("_ingest_ts"))))
  .filter("rn = 1").drop("rn"))
```

Upsert into Delta (SCD Type 1)

```
from delta.tables import DeltaTable
t = DeltaTable.forName(spark, "silver.customers")
(t.alias("t").merge(updates.alias("s"), "t.customer_id = s.customer_id")
   .whenMatchedUpdateAll(condition="t.row_hash <> s.row_hash")
   .whenNotMatchedInsertAll()
   .execute())
```

SCD Type 2 with Delta MERGE (single statement)

```
cur = spark.table("gold.dim_customer").filter("is_current")
changed = (updates.alias("s").join(cur.alias("c"), "customer_id")
           .filter("s.row_hash <> c.row_hash").select("s.*"))

staged = (changed.withColumn("merge_key", F.lit(None).cast("bigint"))          # rows to insert
as new versions
          .unionByName(updates.withColumn("merge_key", F.col("customer_id"))))  # rows to
match/close or insert

(DeltaTable.forName(spark, "gold.dim_customer").alias("t")
  .merge(staged.alias("s"), "t.customer_id = s.merge_key AND t.is_current = true")
  .whenMatchedUpdate(condition="t.row_hash <> s.row_hash",
                     set={"is_current": "false", "valid_to": "s.load_ts"})
  .whenNotMatchedInsert(values={
      "customer_id": "s.customer_id", "city": "s.city", "row_hash": "s.row_hash",
      "valid_from": "s.load_ts", "valid_to": "null", "is_current": "true"})
  .execute())
```

Changed customers appear twice in **staged**: once with their key (to close the old row) and once with a NULL key (never matches, so it inserts the new version). New customers match nothing and insert. Unchanged customers match but fail the update condition.

Incremental processing

```
last = spark.table("silver.orders").agg(F.max("updated_at")).first()[0]
inc = spark.table("bronze.orders")
if last is not None:
    inc = inc.filter(F.col("updated_at") > F.lit(last) - F.expr("INTERVAL 1 HOUR"))   # overlap for late rows
# dedup + MERGE inc
```

Alternatives on Delta: Change Data Feed (**readChangeFeed**), or Structured Streaming with **availableNow** over the bronze table — Spark tracks what was processed in the checkpoint.

Applying CDC events in order

```
w = W.partitionBy("id").orderBy(F.desc("op_ts"))
latest = cdc.withColumn("rn", F.row_number().over(w)).filter("rn = 1")
(target.alias("t").merge(latest.alias("s"), "t.id = s.id")
  .whenMatchedDelete(condition="s.op = 'D'")
  .whenMatchedUpdateAll(condition="s.op <> 'D'")
  .whenNotMatchedInsertAll(condition="s.op <> 'D'")
  .execute())
```

Explode and flatten nested JSON

```
orders = (raw.select(F.from_json("value", schema).alias("o")).select("o.*")
              .select("order_id", "customer.*", F.explode_outer("items").alias("item"))
              .select("order_id", "name", "city", "item.sku", "item.qty", "item.price"))
```

Sessionization

```
w = W.partitionBy("user_id").orderBy("ts")
sessions = (clicks
  .withColumn("gap", F.col("ts").cast("long") - F.lag(F.col("ts").cast("long")).over(w))
  .withColumn("new", F.when(F.col("gap").isNull() | (F.col("gap") > 1800), 1).otherwise(0))
  .withColumn("session_id", F.concat_ws("-", "user_id", F.sum("new").over(w))))
```

Calendar fill

```
bounds = sales.groupBy("store_id").agg(F.min("sale_date").alias("s"), F.max("sale_date").alias("e"))
cal = bounds.select("store_id", F.explode(F.sequence("s", "e")).alias("sale_date"))
full = cal.join(sales, ["store_id", "sale_date"], "left").fillna(0, ["amount"])
```

26. Submitting and deploying Spark jobs

spark-submit

```
spark-submit \
  --master yarn \
  --deploy-mode cluster \
  --name orders_daily \
  --num-executors 10 \
  --executor-cores 4 \
  --executor-memory 16g \
  --driver-memory 8g \
  --conf spark.sql.shuffle.partitions=400 \
  --conf spark.dynamicAllocation.enabled=false \
  --py-files deps.zip \
  --packages io.delta:delta-spark_2.12:3.2.0 \
  main.py --run-date 2026-01-15
```

Packaging Python code

- Small projects: **--py-files** with a zip of your package.
- Better: build a wheel and install it on the cluster (Databricks libraries, cluster init scripts, or a custom image).
- Third-party Python libraries must be installed on **every** executor, not only the driver.

Job structure for production

```
def main(run_date: str):
    spark = SparkSession.builder.appName(f"orders_{run_date}").getOrCreate()
    raw = extract(spark, run_date)
    clean = transform(raw)
    validate(clean)
    load(clean, run_date)          # idempotent: replaceWhere / MERGE

if __name__ == "__main__":
    import sys
    main(sys.argv[sys.argv.index("--run-date") + 1])
```

- Parameterize by run date; never use **current_date()** for which data to process.
- Keep transforms as functions of DataFrames → DataFrames, so they're testable with small local data.
- Make the load idempotent, so the orchestrator can retry safely.
- Log row counts and key metrics at each step.

The core habit for Spark: think about **how data moves**. Before running a job on big data, look at the plan — how many shuffles, which join strategies, how many partitions — and ask if each shuffle is really needed.

Sagar

27. Common beginner mistakes

Almost everyone makes these when starting with Spark. Knowing them saves days of debugging.

Sagar

Mistake	What happens	Do this instead
Treating Spark like pandas: `collect()` / `toPandas()` on big data	All data goes to the driver → driver out of memory	Keep data distributed; write results, or `limit()` before collecting
Looping over rows in Python (`for row in df.collect()`)	Slow, single-machine, defeats Spark	Express logic with DataFrame functions, joins and windows
Using Python UDFs for simple logic	5–10× slower, no optimization	`when/otherwise`, `regexp_replace`, built-in functions
`inferSchema=True` in production	Extra full read; wrong types (IDs lose leading zeros)	Explicit schema
`mode("overwrite")` on a partitioned table for one day	Whole table replaced with one day	Dynamic partition overwrite or Delta `replaceWhere`
`union` instead of `unionByName`	Columns silently mixed up when order differs	`unionByName(..., allowMissingColumns=True)`
`and` / `or` / `not` in filters	Python error or wrong logic	`&`, `|`, `~` with parentheses around each condition
`dropDuplicates(["id"])` to "keep the latest"	Keeps an arbitrary row	`row_number()` over a window ordered by timestamp
Calling `count()` everywhere for logging	Each count is a full job on the whole data	Count once, or compute several metrics in one `agg`
`coalesce(1)` before writing big output	One task does all the work, very slow	Repartition to sensible file sizes
Caching everything	Memory pressure, spills, OOM	Cache only reused, expensive DataFrames, then `unpersist()`
Ignoring the Spark UI	Guessing at performance problems	Check skew, spill, shuffle size and task times first
`withColumn` in a loop over hundreds of columns	Huge plans, slow analysis	One `select` with a list of expressions
Joining on columns with different types	Implicit casts, wrong or missing matches	Cast once so both sides match
Not handling NULL join keys	NULLs never match; many NULLs create a skewed partition	Filter or handle NULL keys separately
`datetime.now()` / `current_date()` to decide which data to process	Reruns and backfills process the wrong day	Pass `run_date` as a parameter
Deleting or sharing a streaming checkpoint	Reprocessing, duplicates, or broken queries	One checkpoint per query, never touched casually

Most of these come from the same root cause: forgetting that Spark is distributed. Ask two questions about every line you write: where does this run (driver or executors)? and does this move data between machines?

sagar

28. Cluster sizing scenarios: 10 GB to 2 TB per day

Sizing a Spark cluster is an estimate you make from a few numbers, then confirm with a test run and the Spark UI. Interviewers mainly want to see that you have a **method** and know which numbers matter.

Step 1: write down the assumptions

Assumption	Value used in these scenarios
Daily input	10 GB, 100 GB, 500 GB, 1 TB, 2 TB of **compressed Parquet/Delta**
Job shape	Read → clean/filter → join with dimensions (broadcast where small) → aggregate → write Delta. About **3 heavy stages**
Input partition size	128 MB (**spark.sql.files.maxPartitionBytes**)
Time per task	~1 minute to process a 128 MB partition through a heavy stage (typical ETL; simpler jobs or Photon are faster)
Target runtime	~10–40 minutes depending on size (the batch window / SLA)
Shuffle size	About the same as the input (join + aggregation), target ~200 MB per shuffle partition
Worker node	**16 vCPU, 128 GB RAM** (memory-optimized, e.g. Azure E16s_v5 / AWS r5.4xlarge)

> If the input is raw CSV/JSON instead of Parquet, the same data is several times larger on disk and parsing costs more CPU. Convert to Parquet/Delta in the bronze step, then size the heavy jobs on the Parquet size.

Step 2: executor layout per node

Item	Calculation	Result
Usable cores per node	16 − 1 (OS, NodeManager/daemons)	**15**
Cores per executor	Rule of thumb 4–5 (good HDFS/ABFS throughput, less GC pressure)	**5**
Executors per node	15 / 5	**3**
Usable memory per node	128 GB − ~8 GB (OS, daemons)	**~120 GB**
Memory per executor container	120 / 3	**40 GB**
memoryOverhead	max(384 MB, 10% of heap)	**~4 GB**
spark.executor.memory (heap)	40 − 4	**36 GB**
Unified (execution + storage) memory	(36 GB − 300 MB) × 0.6	**~21 GB per executor ≈ 4.3 GB per running task**

4.3 GB per task is plenty for 128–200 MB partitions, even after they expand 3–4× in memory. That headroom is what avoids spills and OOMs.

Step 3: the formulas

```
input_partitions   = input_size / 128 MB
core_minutes       = input_partitions × heavy_stages × minutes_per_task
cores_needed       = core_minutes / target_runtime_minutes
executors          = cores_needed / 5
worker_nodes       = executors / 3            (15 usable cores per node)
waves              = input_partitions / total_cores     (aim for a handful, not
100s)
shuffle_partitions = shuffle_size / ~200 MB   (round to a multiple of total cores)
```

Step 4: the five scenarios

Daily input	Input tasks (÷128 MB)	Target runtime	Cores needed	Worker nodes (16c/128 GB)	Executors (5 cores, 36 GB)	Total cores	Waves per stage	Shuffle partitions	Driver
10 GB	80	~10 min	24	**2**	6	30	~3	64–100 (or AQE from 200)	4 cores, 8 GB
100 GB	800	~30 min	80	**6**	18	90	~9	500–720	8 cores, 16 GB
500 GB	4,000	~40 min	300	**20**	60	300	~13	2,500–3,000	8 cores, 16–32 GB
1 TB	8,000	~40 min	600	**40**	120	600	~13	5,000–6,000	16 cores, 32 GB
2 TB	16,000	~40 min	1,200	**80**	240	1,200	~13	10,000–12,000	16 cores, 32–64 GB

How to read the trade-off: cores × time is roughly constant. If the SLA for 1 TB is 80 minutes instead of 40, ~20 nodes are enough; if it must finish in 20 minutes, you need ~80 nodes (and the job must have enough partitions to use them).

Worked example: 1 TB per day, 40-minute window

1. **Input partitions:** 1,000 GB ÷ 128 MB ≈ 8,000 tasks per heavy stage.
2. **Work:** 8,000 tasks × 3 heavy stages × ~1 min = 24,000 core-minutes.
3. **Cores:** 24,000 ÷ 40 min = **600 cores**.

Sagar

4. **Executors:** 600 ÷ 5 = **120 executors** of 5 cores / 36 GB heap / 4 GB overhead.
5. **Nodes:** 120 ÷ 3 = **40 worker nodes** of 16 vCPU / 128 GB.
6. **Waves:** 8,000 ÷ 600 ≈ 13 waves per stage, ~13 minutes per stage, ~40 minutes total.
7. **Shuffle partitions:** ~1 TB shuffle ÷ 200 MB ≈ 5,000 → round to a multiple of 600 cores → **6,000** (with AQE coalescing on).
8. **Shuffle disk:** ~1 TB of shuffle files spread over 40 nodes ≈ 25 GB per node — make sure local disks (ideally SSD) have room, plus spill headroom.
9. **Driver:** 16 cores / 32 GB — planning 8,000+ tasks and broadcasting dimensions needs more than the default.

```
spark-submit \
  --master yarn --deploy-mode cluster \
  --num-executors 120 \
  --executor-cores 5 \
  --executor-memory 36g \
  --conf spark.executor.memoryOverhead=4g \
  --driver-cores 16 --driver-memory 32g \
  --conf spark.sql.shuffle.partitions=6000 \
  --conf spark.sql.adaptive.enabled=true \
  --conf spark.sql.adaptive.coalescePartitions.enabled=true \
  --conf spark.sql.adaptive.skewJoin.enabled=true \
  --conf spark.sql.autoBroadcastJoinThreshold=100m \
  orders_daily.py --run-date 2026-01-15
```

On Databricks you'd pick ~40 workers of a 16-core/128 GB node type (or autoscale 20–50), and let Databricks set executor layout; keep AQE on and set shuffle partitions explicitly or to **auto**.

The classic interview calculation

"You have 10 nodes, each with 16 cores and 64 GB RAM. How many executors, and with what settings?"

Step	Calculation	Result
Usable per node	16 − 1 core, 64 − 1 GB	15 cores, 63 GB
Executors per node	15 cores ÷ 5	3
Total executors	10 × 3 = 30, minus 1 for the YARN ApplicationMaster/driver	**29**
Memory per executor	63 GB ÷ 3	21 GB
Overhead	~7–10% of 21 GB	~2 GB
`--executor-memory`	21 − 2	**~19 GB**
Final		`--num-executors 29 --executor-cores 5 --executor-memory 19g`

Why not 1 executor with 15 cores per node? Large heaps mean long GC pauses and poor HDFS/ABFS throughput per JVM. Why not 15 executors with 1 core? No parallelism inside a JVM, broadcast variables copied 15 times per node, and per-JVM overhead wasted. 4–5 cores per executor is the middle ground.

What changes the numbers

Factor	Effect on sizing
Skew	A single hot key makes one task run for an hour — more nodes won't help. Fix skew first (AQE, salting, broadcast)
Heavy Python UDFs	More CPU per task, more memoryOverhead for Python workers
Wide joins without broadcast	Larger shuffles → more shuffle partitions, more disk
Caching large DataFrames	Needs more storage memory — or a smaller cache
Raw JSON/CSV	More CPU and bytes than Parquet; parse once into bronze
Photon / vectorized engines	Fewer cores for the same SLA
Incremental processing	Processing only new/changed data shrinks every number above
Spot instances	Cheaper workers; keep the driver on-demand and expect occasional task retries

Validate, don't guess

1. Run the job on a sample (for example one hour of data, ~4% of a day) on a small cluster.
2. Read the Spark UI: task durations, shuffle sizes, spill, GC time, skew.

3. Scale linearly to the full day and to the SLA, then add ~20–30% headroom for growth and peak days.
4. Turn on autoscaling with a sensible max so peak days don't fail and normal days don't overpay.
5. Re-check monthly — data volume grows.

Same volumes as a stream

Spread across 24 hours, the same data needs far less compute than a short batch window:

Daily volume	Average rate	Peak (≈3× average)	Kafka partitions (≈10 MB/s each, with headroom)	Typical streaming cluster
10 GB	~0.12 MB/s	~0.4 MB/s	3–6	1–2 small workers
100 GB	~1.2 MB/s	~3.5 MB/s	6–12	2 workers
500 GB	~5.8 MB/s	~17 MB/s	12–24	2–4 workers
1 TB	~11.6 MB/s	~35 MB/s	24–48	4–6 workers
2 TB	~23 MB/s	~70 MB/s	48–96	6–10 workers

Streaming trades a big short burst for a small always-on cluster. The right choice depends on the latency the business needs, the cost of 24/7 compute, and operational effort — not just volume.

> In an interview, say the method out loud: partitions from data size → work from tasks × stages × time per task → cores from the SLA → executors of 5 cores → nodes from executors per node → memory per executor with overhead → shuffle partitions from shuffle size. Then mention skew, AQE, a sample run and autoscaling. That's what separates a real answer from a memorized number.

29. Practice project: an end-to-end mini pipeline

Build this on your laptop or in Databricks Free Edition. It uses almost every concept in the handbook on data you generate yourself.

The scenario

*An online store has **orders** (with duplicates and late updates) and **customers**. Build bronze → silver → gold tables and answer business questions.*

Step 1: generate raw data

```
from pyspark.sql import SparkSession, functions as F, Window as W
spark = SparkSession.builder.master("local[*]").appName("practice").getOrCreate()

customers = spark.createDataFrame(
    [(1, "Asha", "Bengaluru"), (2, "Ravi", "Pune"), (3, "Meena", "Chennai"), (4, "Arjun", None)],
    "customer_id INT, name STRING, city STRING")

orders = (spark.range(1, 200_001).withColumnRenamed("id", "order_id")
  .withColumn("customer_id", (F.rand(seed=1) * 4 + 1).cast("int"))
  .withColumn("amount", F.round(F.rand(seed=2) * 5000, 2))
  .withColumn("status", F.element_at(F.array(F.lit("PLACED"), F.lit("SHIPPED"),
F.lit("DELIVERED"), F.lit("CANCELLED")),
                                     (F.rand(seed=3) * 4 + 1).cast("int")))
  .withColumn("updated_at", F.expr("timestamp('2026-01-01') + make_interval(0,0,0,0,0,0,
cast(rand(4) * 2592000 as int))")))

dupes = orders.sample(fraction=0.05, seed=5).withColumn("updated_at", F.col("updated_at") +
F.expr("INTERVAL 1 HOUR"))
raw = orders.unionByName(dupes)                          # ~5% of orders have a later second
version
raw.write.mode("overwrite").json("practice/landing/orders")
customers.write.mode("overwrite").json("practice/landing/customers")
```

Step 2: bronze — read raw with an explicit schema

```
order_schema = "order_id LONG, customer_id INT, amount DOUBLE, status STRING, updated_at
TIMESTAMP"
bronze = (spark.read.schema(order_schema).json("practice/landing/orders")
          .withColumn("_ingest_ts", F.current_timestamp())
          .withColumn("_source_file", F.input_file_name()))
bronze.write.mode("overwrite").parquet("practice/bronze/orders")
```

Step 3: silver — clean, deduplicate (latest version wins), add date

```
w = W.partitionBy("order_id").orderBy(F.desc("updated_at"))
silver = (spark.read.parquet("practice/bronze/orders")
  .filter(F.col("order_id").isNotNull())
  .withColumn("amount", F.col("amount").cast("decimal(18,2)"))
  .withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")
  .withColumn("order_date", F.to_date("updated_at")))

spark.conf.set("spark.sql.sources.partitionOverwriteMode", "dynamic")
silver.repartition("order_date").write.mode("overwrite").partitionBy("order_date").parquet("practice/silver/orders")
```

Step 4: gold — business tables

```
cust = spark.read.schema("customer_id INT, name STRING, city
STRING").json("practice/landing/customers") \
            .fillna({"city": "UNKNOWN"})
s = spark.read.parquet("practice/silver/orders")

daily_city = (s.filter("status <> 'CANCELLED'")
  .join(F.broadcast(cust), "customer_id", "left")
  .groupBy("order_date", "city")
  .agg(F.count("*").alias("orders"), F.sum("amount").alias("revenue")))

running = daily_city.withColumn("running_revenue",
    F.sum("revenue").over(W.partitionBy("city").orderBy("order_date")
                          .rowsBetween(W.unboundedPreceding, 0)))
running.write.mode("overwrite").parquet("practice/gold/daily_city_revenue")
```

Step 5: check your work

```
assert s.groupBy("order_id").count().filter("count > 1").count() == 0, "duplicates in silver"
print("bronze rows:", spark.read.parquet("practice/bronze/orders").count())
print("silver rows:", s.count())          # should be 200,000
spark.read.parquet("practice/gold/daily_city_revenue").orderBy("city", "order_date").show(10)
```

Step 6: questions to answer yourself

1. Top 3 customers by revenue in each city.
2. Month-over-month revenue growth % per city.
3. Customers with no orders in the last 7 days of the data.
4. Percentage of orders cancelled per day.
5. Open the Spark UI: how many stages did step 3 create, and why? Where are the shuffles?
6. Run **`explain(mode="formatted")`** on step 4. Is the customers join a broadcast join?
7. Change step 3 to use **`dropDuplicates(["order_id"])`** and compare totals — why can the result be wrong?
8. Rewrite steps 2–3 as a Structured Streaming job with **`trigger(availableNow=True)`** reading new files from the landing folder.

Finish this project and you've used schemas, file formats, partitioned writes, deduplication with windows, broadcast joins, aggregations, running totals, data-quality checks and the Spark UI — the same toolkit as a real pipeline.

sagar

30. PySpark cheat sheet

Session, reading and writing

```
from pyspark.sql import SparkSession, functions as F, Window as W
spark = SparkSession.builder.appName("job").getOrCreate()

df = spark.read.option("header", True).schema(schema).csv(path)
df = spark.read.json(path); df = spark.read.parquet(path); df = spark.table("db.t")
df = spark.read.format("delta").option("versionAsOf", 5).load(path)
df = (spark.read.format("jdbc").option("url", url).option("dbtable", "t")
      .option("partitionColumn", "id").option("lowerBound", 1).option("upperBound", 10**7)
      .option("numPartitions", 16).load())

df.write.mode("overwrite").partitionBy("dt").parquet(path)          # append|overwrite|ignore|error
df.write.format("delta").mode("append").saveAsTable("db.t")
df.write.format("delta").mode("overwrite").option("replaceWhere", "dt='2026-01-
15'").saveAsTable("db.t")
spark.conf.set("spark.sql.sources.partitionOverwriteMode", "dynamic")
```

Columns and rows

```
df.select("a", F.col("b").alias("bb"), (F.col("c") * 2).alias("c2"))
df.withColumn("x", F.lit(1)); df.withColumns({"x": F.lit(1), "y": F.col("a") + 1})
df.withColumnRenamed("old", "new"); df.drop("a", "b")
df.filter((F.col("a") > 10) & F.col("b").isNotNull()); df.where("a > 10")
df.withColumn("band", F.when(F.col("amt") > 1000, "high").otherwise("low"))
df.dropDuplicates(["id"]); df.distinct(); df.orderBy(F.desc("ts")); df.limit(10)
df.unionByName(df2, allowMissingColumns=True)
df.fillna({"city": "NA"}); df.dropna(subset=["id"]); F.coalesce("a", "b")
F.col("amt").cast("decimal(18,2)"); F.col("a").isin(1, 2); F.col("s").rlike("^A")
```

Functions you use daily

```
F.trim, F.lower, F.upper, F.concat_ws("-", "a", "b"), F.split("s", ","), F.regexp_replace("p", "[^0-9]",
"")
F.regexp_extract("e", "@(.+)", 1), F.substring("s", 1, 3), F.lpad("id", 6, "0"), F.sha2(F.concat_ws("|",
*cols), 256)
F.to_date("s", "dd-MM-yyyy"), F.to_timestamp("s"), F.date_format("ts", "yyyy-MM"),
F.date_add("d", 1)
F.datediff("end", "start"), F.trunc("d", "month"), F.date_trunc("hour", "ts"), F.current_date()
F.from_utc_timestamp("ts", "Asia/Kolkata"), F.unix_timestamp("ts"), F.sequence("start", "end")
F.from_json("v", schema), F.to_json(F.struct("*")), F.get_json_object("v", "$.id")
F.explode("arr"), F.explode_outer("arr"), F.size("arr"), F.array_contains("arr", "x")
F.collect_list("x"), F.collect_set("x"), F.transform("arr", lambda x: x * 2)
```

Aggregations, joins, windows

```
df.groupBy("k").agg(F.count("*").alias("n"), F.sum("amt"), F.countDistinct("u"),
                    F.approx_count_distinct("u"), F.percentile_approx("amt", 0.5))
df.groupBy("p").pivot("m", ["Jan", "Feb"]).sum("amt")

a.join(b, "id", "left")                          # inner|left|right|full|left_semi|left_anti|cross
a.join(F.broadcast(dim), "id")
a.alias("x").join(b.alias("y"), F.col("x.id") == F.col("y.id"))

w = W.partitionBy("k").orderBy(F.desc("ts"))
df.withColumn("rn", F.row_number().over(w)).filter("rn = 1")          # latest per key
F.rank(), F.dense_rank(), F.lag("x"), F.lead("x"), F.ntile(4)
F.sum("amt").over(W.partitionBy("k").orderBy("d").rowsBetween(W.unboundedPreceding, 0))   # running total
F.avg("amt").over(W.partitionBy("k").orderBy("d").rowsBetween(-6, 0))                    # 7-row moving avg
F.last("v", ignorenulls=True).over(W.partitionBy("k").orderBy("ts").rowsBetween(W.unboundedPreceding, 0))  # ffill
```

Partitions, caching, plans

```
from pyspark import StorageLevel
df.rdd.getNumPartitions()
df.repartition(200); df.repartition("dt"); df.coalesce(10)
df.cache(); df.persist(StorageLevel.MEMORY_AND_DISK); df.unpersist()
df.explain(mode="formatted")
df.groupBy(F.spark_partition_id()).count()          # rows per partition → skew check
spark.conf.set("spark.sql.shuffle.partitions", "400")
```

Delta and streaming

```
from delta.tables import DeltaTable
(DeltaTable.forName(spark, "t").alias("t").merge(s.alias("s"), "t.id = s.id")
   .whenMatchedUpdateAll().whenNotMatchedInsertAll().execute())
spark.sql("OPTIMIZE t ZORDER BY (id)"); spark.sql("VACUUM t RETAIN 168 HOURS")
spark.sql("DESCRIBE HISTORY t"); spark.sql("RESTORE TABLE t TO VERSION AS OF 10")

s = (spark.readStream.format("kafka").option("kafka.bootstrap.servers", b)
      .option("subscribe", "topic").option("startingOffsets", "latest").load())
(s.withWatermark("ts", "10 minutes").groupBy(F.window("ts", "5 minutes")).count()
  .writeStream.format("delta").outputMode("append")
  .option("checkpointLocation", "/chk/q").trigger(availableNow=True).toTable("t"))
stream.writeStream.foreachBatch(lambda bdf, bid: merge(bdf)).option("checkpointLocation",
c).start()
```

Configs worth knowing

Config	Default	Why
`spark.sql.shuffle.partitions`	200	Tasks after a shuffle
`spark.sql.adaptive.enabled`	true (3.2+)	AQE: coalesce, skew join, join switching
`spark.sql.autoBroadcastJoinThreshold`	10 MB	Auto broadcast limit (-1 disables)
`spark.sql.files.maxPartitionBytes`	128 MB	Input partition size
`spark.sql.sources.partitionOverwriteMode`	static	**dynamic** to overwrite only written partitions
`spark.executor.memory` / `.cores` / `.memoryOverhead`	cluster-specific	Executor sizing
`spark.dynamicAllocation.enabled`	false (varies)	Scale executors with load
`spark.sql.execution.arrow.pyspark.enabled`	false (varies)	Faster pandas conversion

Sagar

Quick answers to common questions

Question	Short answer
RDD vs DataFrame vs Dataset	RDD: objects, no schema, no optimizer; DataFrame: schema + Catalyst/Tungsten, default; Dataset: typed DataFrame, Scala/Java only
repartition vs coalesce	repartition = full shuffle, can increase, even sizes; coalesce = no shuffle, only decrease, can be uneven
cache vs persist	cache = persist(MEMORY_AND_DISK) for DataFrames; persist lets you choose the level
narrow vs wide	narrow: no data movement (filter, select); wide: shuffle (groupBy, join, distinct)
Transformation vs action	transformations are lazy and build the plan; actions (count, write, collect) run jobs
Job / stage / task	action → job; shuffle boundaries → stages; one task per partition per stage
Why is a job slow?	Compare runs in Spark UI: input growth, skew (max ≫ median task), spill, small files, too few/many partitions, Python UDFs
Fix skew	AQE skew join, broadcast the small side, isolate hot keys, salting
Fix executor OOM	More/smaller partitions, fix skew, less caching, more memoryOverhead for Python, then more memory
Fix driver OOM	Avoid collect/toPandas on big data, don't broadcast big tables, compact small files
Small files	repartition by partition column before write, coalesce, Delta OPTIMIZE / auto-compaction
Broadcast join limit	Default 10 MB auto; hint **broadcast()** up to what executors/driver can hold
dropDuplicates vs row_number	dropDuplicates keeps an arbitrary row; row_number + order lets you choose (latest)
union vs unionByName	union matches by position; unionByName by column name
Exactly-once streaming	Replayable source + checkpoint + idempotent sink (Delta / MERGE)
Watermark	Bound on lateness; lets Spark finalize windows and drop old state
Executor sizing rule	~5 cores per executor, leave 1 core + some GB per node for OS, ~10% memoryOverhead
Shuffle partition sizing	~100–200 MB per partition; let AQE coalesce
UDF performance	built-in functions > pandas UDF > Python UDF

31. Remember it for life: memory hooks

Facts on their own fade within days. What we remember are ***stories, places, people, pictures and patterns****. This chapter turns Spark into exactly those, so a quick look brings the whole idea back.*

Why these tricks work

Memory principle	What it means	How this chapter uses it
Stories and analogies	The brain stores meaning by linking new ideas to things it already knows	Spark as a restaurant kitchen — everyone knows how a kitchen works
Method of loci (memory palace)	Placing ideas at locations along a familiar walk makes them easy to recall in order	A walk through the kitchen, from the entrance to the storeroom
Chunking	We hold only about 4 items at a time, so group details into small chunks	Everything grouped into threes and fours
Mnemonics	First letters or rhymes act as a hook to pull out a list	*Don't Cook Everything Together, Really Careful Ingestion*
Vivid, emotional images	Funny, strange or physical images stick better than abstract words	The lazy genius, the overloaded cook, the overflowing fridge
Dual coding	A picture plus words is remembered better than words alone	The architecture diagram plus the kitchen story
Retrieval practice	Testing yourself beats re-reading	The cover-and-recall self-test at the end
Spaced repetition	Reviewing just before you'd forget locks memory in	The 1–3–7–21 day revision plan

The story: a night in the Spark Kitchen

Read this once slowly and picture each scene. Later, just walking through the kitchen in your head brings back every concept.

1. **The entrance — the order ticket.** *A waiter writes down orders all evening, but the kitchen does nothing until the ticket is finally sent in. Transformations are written orders; the action is sending the ticket.* (**Lazy evaluation**)
2. **The head chef's desk.** *The head chef reads the whole ticket, plans the dishes and writes a step-by-step plan. The head chef is the driver; the plan is the DAG.*
3. **The manager's office.** *The head chef asks the restaurant manager for cooks. The manager hires them and sends them to the stations, then goes back to the office — the manager never cooks. The manager is the cluster manager; the cooks are executors.*
4. **The stations.** *Vegetables arrive in baskets. Each cook takes one basket at a time and works on it. Baskets are partitions; one cook on one basket is a task; several rounds of baskets are waves.*

5. **Between courses — the walk.** Before the main course, every cook must carry their chopped onions to the onion station, their tomatoes to the tomato station. It's slow, noisy and everyone bumps into each other. *That walk is the shuffle, and each course is a stage.*
6. **The spice box.** A tiny spice box is copied and placed at every station, so no one walks to fetch spices. *That's a broadcast join.*
7. **The overloaded cook.** One cook got a basket ten times bigger than everyone else's. The whole kitchen waits for them. *That's skew. Fix: split the big basket (salting), or let the kitchen manager rebalance it (AQE).*
8. **The counter overflowing onto the floor.** When a cook's counter is full, they put food on the floor and pick it up later — it works, but slowly. *That's spill to disk; too much spilling means baskets are too big.*
9. **The fridge.** Dishes needed again later go in the fridge so they don't have to be cooked twice. *That's cache/persist — and a full fridge with unused food just wastes space.*
10. **The recipe card.** If a cook drops a dish, they don't panic — the recipe card says exactly how to remake it. *That's lineage and fault tolerance.*
11. **The live adjustments.** Halfway through service, the kitchen manager sees one station is idle and another is swamped, and moves work around. *That's Adaptive Query Execution.*
12. **The delivery counter.** Large orders go straight out of the back door to customers; only the bill comes back to the head chef. *Executors write output directly to storage; only small results go to the driver.*
13. **The night shift logbook.** For delivery orders arriving all night, the kitchen keeps a logbook: "last order cooked: #4,512". After a power cut, it restarts from #4,513. *That's Structured Streaming with a checkpoint.*
14. **The last-entry rule.** At closing, guests arriving more than 15 minutes late aren't seated, so the kitchen can finally close the evening's bill. *That's a watermark.*

One-look concept anchors

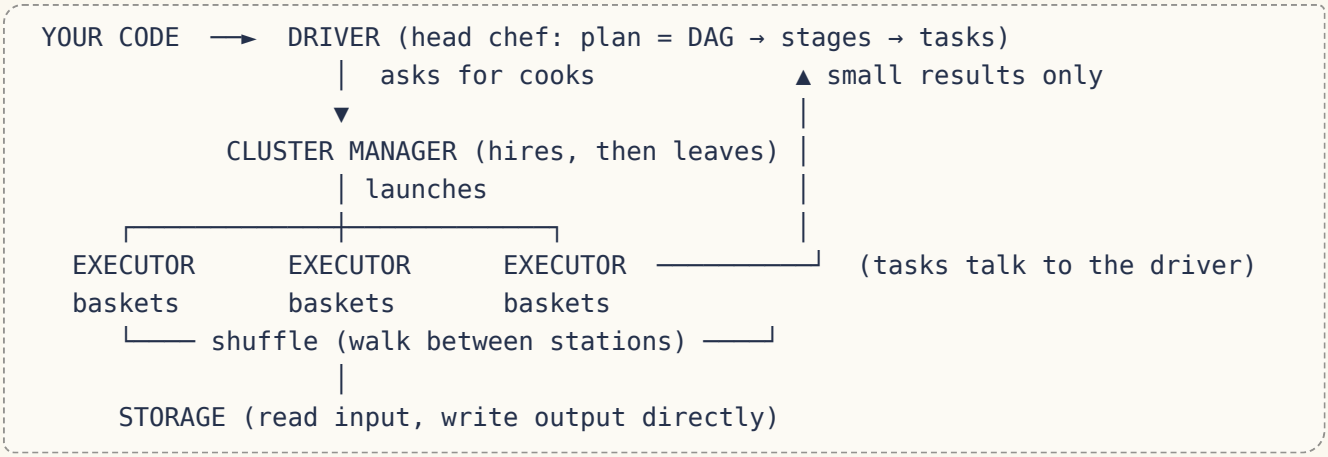
Concept	Picture it as	One-line hook
Driver	Head chef with the plan	Plans and schedules, doesn't chop
Executor	A cook at a station	Does the real work, holds the fridge
Cluster manager	Restaurant manager	Hires cooks, then steps away
Partition	A basket of vegetables	Goldilocks size: ~128–200 MB
Task	One cook, one basket	One task per partition per stage
Stage	A course of the meal	A shuffle ends a course
Job	One full order ticket	One action = one job
Lazy evaluation	A lazy genius	Plans everything, does nothing until asked
Shuffle	Everyone walking ingredients across the kitchen	The most expensive thing Spark does
Broadcast join	A spice box at every station	Small table? Spread it
Sort-merge join	Both teams lining up in order to pair off	Both big? Sort and merge
Skew	One overloaded cook	One slow task holds up the stage
Salting	Splitting the giant basket into smaller ones	Spread the hot key
Spill	Food on the floor	Memory full → disk → slow
Cache	The fridge	Store only what you'll reuse
Lineage	The recipe card	Lost work is rebuilt, not lost
Catalyst	GPS planning the route	Rewrites your query before running
AQE	Live traffic updates	Re-plans during the run
Checkpoint	The night-shift logbook	Restart exactly where you stopped
Watermark	The last-entry time	How late is too late
RDD	Unlabelled sacks of ingredients	Flexible, but the chef can't plan
DataFrame	Labelled containers on a spreadsheet	The chef can see and optimize everything
Dataset	Labelled and locked containers	Typed — Scala/Java only

Mnemonics cheat card

Mnemonic	Stands for
D·**C**·**E**·**T** — *Don't Cook Everything Together*	**D**river, **C**luster manager, **E**xecutor, **T**ask
J·**S**·**T** — *Just Split Tasks*	*1 action → 1* **J**ob; *1 shuffle → new* **S**tage; *1 partition → 1* **T**ask
Can Charlie Show The Work First?	Actions: **C**ount, **C**ollect, **S**how, **T**ake, **W**rite, **F**irst
Just Getting Data Organized Requires a shuffle	Wide ops: **J**oin, **G**roupBy, **D**istinct, **O**rderBy, **R**epartition
Narrow stays in its Lane, Wide Walks	Narrow = no data movement; wide = shuffle
Repartition = Reshuffle, Coalesce = Combine	repartition: full shuffle, up or down; coalesce: no shuffle, only down
Cache is Common, Persist lets you Pick	cache = default level; persist = choose the storage level
B·**A**·**S**·**H** *the skew*	**B**roadcast, **A**QE skew join, **S**alting, **H**andle hot keys separately
R·**C**·**I** — *Really Careful Ingestion*	Exactly-once = **R**eplayable source + **C**heckpoint + **I**dempotent sink
5 · 1 · 10	5 cores per executor, leave 1 core per node, ~10% memory overhead
Built-in Beats Pandas Beats Python	UDF preference: built-in functions > pandas UDF > Python UDF
A·**O**·**I**·**E** — *Always Overwrite Intentionally, Everyone*	Save modes: **A**ppend, **O**verwrite, **I**gnore, **E**rror
Read less, Move less, Balance, No Python	The four performance rules: prune/filter, avoid shuffles, fix skew/partitions, avoid UDFs
To the boss vs too heavy a basket	Driver OOM = data sent **to** the driver; executor OOM = a partition too big

Spark on a napkin

If you can redraw this from memory, you understand Spark:



Self-test: cover the right column

Cover the answers, say each one out loud, then check. Saying it beats reading it.

Question	Answer
What happens when you call `filter()`?	Nothing runs — it's added to the plan (lazy)
What starts a job?	An action (count, collect, show, take, write)
What splits a job into stages?	A shuffle
How many tasks in a stage?	One per partition
Who processes the data — driver or executors?	Executors
Does the cluster manager move your data?	No — it only allocates resources
Cheapest way to join a 50 MB table to a 2 TB table?	Broadcast the 50 MB table
One task runs 20× longer than the rest — why?	Skew
Four ways to fix skew?	Broadcast, AQE, salting, handle hot keys separately
repartition vs coalesce?	Reshuffle (up/down, full shuffle) vs combine (down only, no shuffle)
When should you cache?	When a DataFrame is reused by several actions and expensive to recompute
Three parts of exactly-once streaming?	Replayable source, checkpoint, idempotent sink
What does a watermark do?	Limits lateness so windows can close and old state is dropped
Typical executor size?	~5 cores, leave 1 core per node for the OS, ~10% memory overhead
Driver OOM — most common cause?	collect/toPandas or broadcasting a big table

Revision plan (spaced repetition)

When	What to do	Time
Day 1	Read the kitchen story; redraw "Spark on a napkin" from memory	20 min
Day 3	Self-test table; re-read only the questions you missed	10 min
Day 7	Explain Spark architecture and a shuffle out loud to a friend (or record yourself)	10 min
Day 21	Mnemonics card + self-test; solve two PySpark problems from memory	20 min
Before an interview	Kitchen story + napkin drawing + cheat sheet	15 min

> The fastest way to own a concept is to teach it. If you can explain Spark using the kitchen story to someone who has never coded, you will never forget it — and you'll explain it clearly in any interview.

Sagar

32. Glossary

Plain-language definitions of the terms used in this handbook.

Sagar

Term	Meaning
Action	An operation that returns a result or writes data (**count**, **show**, **collect**, **write**) and makes Spark actually run
AQE (Adaptive Query Execution)	Spark re-optimizing a query while it runs, using real data sizes (merging small partitions, fixing skew, switching to broadcast joins)
Broadcast join	Copying a small table to every executor so the big table doesn't need to be shuffled
Bucketing	Pre-splitting a table into a fixed number of files by hashing a column, to speed up joins on that column
Cache / persist	Keeping a computed DataFrame in memory (and/or disk) so later actions reuse it
Catalyst	Spark's query optimizer that rewrites your plan into a faster one
Checkpoint (streaming)	Folder where a streaming query stores its progress (offsets, state) so it can restart where it stopped
Cluster manager	System that gives Spark machines/containers: YARN, Kubernetes, Standalone, Databricks
Coalesce	Reducing the number of partitions without a full shuffle
Column pruning	Reading only the columns a query needs from columnar files
DAG	Directed Acyclic Graph — the plan of stages and their dependencies
DataFrame	A distributed table with named, typed columns — the main PySpark API
Delta Lake	Table format that adds ACID transactions, MERGE, time travel and schema enforcement to Parquet files
Driver	The process running your program; plans the work and schedules tasks
Dynamic partition overwrite	Overwriting only the partitions present in the output, not the whole table
Executor	A worker process on a cluster node that runs tasks and stores cached data
Explain plan	Text showing how Spark will execute a query (**df.explain()**)
Idempotent	Running the same job twice gives the same result as running it once — safe reruns
Lazy evaluation	Transformations only build a plan; work happens at an action
Lineage	The recorded chain of transformations; used to recompute lost partitions
memoryOverhead	Extra memory outside the JVM heap for Python workers, buffers and JVM overhead
Narrow transformation	Each output partition needs only one input partition (filter, select) — no shuffle
Partition	A chunk of data processed by one task

Term	Meaning
Partition pruning	Skipping folders/partitions that a filter rules out
Predicate pushdown	Sending filters down to the file reader so it can skip data using file statistics
Repartition	Changing the number of partitions (or partitioning by a column) with a full shuffle
RDD	Resilient Distributed Dataset — Spark's original low-level API of distributed objects
Salting	Adding a random number to a skewed key to spread its rows over several partitions
Schema	Column names and data types of a DataFrame
Shuffle	Redistributing data across executors by key; needed for joins, groupBy, distinct, orderBy
Skew	Uneven data distribution — a few keys or partitions much larger than the rest, so a few tasks run far longer
Spill	Writing data from memory to disk because a task ran out of execution memory
Stage	A group of tasks that run without a shuffle; shuffles split jobs into stages
Structured Streaming	Spark's streaming engine that runs DataFrame queries incrementally on new data
Task	The smallest unit of work: one partition processed by one core in one stage
Transformation	An operation that defines a new DataFrame (select, filter, join) without running anything yet
Trigger	When a streaming query processes the next batch (every N minutes, once, availableNow)
Tungsten	Spark's execution engine improvements: compact binary memory format, code generation
UDF	User-Defined Function written in Python/Scala; slower than built-in functions
Watermark	A limit on how late events can arrive in a stream, so Spark can finalize windows and drop old state
Wave	One round of tasks when there are more partitions than cores (1,000 tasks on 100 cores = 10 waves)
Wide transformation	Output partitions need data from many input partitions — causes a shuffle
Window function	Computes a value for each row from related rows (rank, lag, running total) without collapsing rows

Learn these words well: in interviews and design discussions, explaining a problem with the right term — "the join stage is skewed and spilling" — immediately shows you understand Spark.

— Sagar

Sagar